



11. 循环神经网络-2



- Andrew Ng, Recurrent Neural Network

<https://www.bilibili.com/video/av88010803?p=1>

- 李宏毅，深度学习

<https://www.bilibili.com/video/BV1JE411g7XF>

建设单位： 大连理工大学

建设人： 孔雨秋

目录

CONTENT

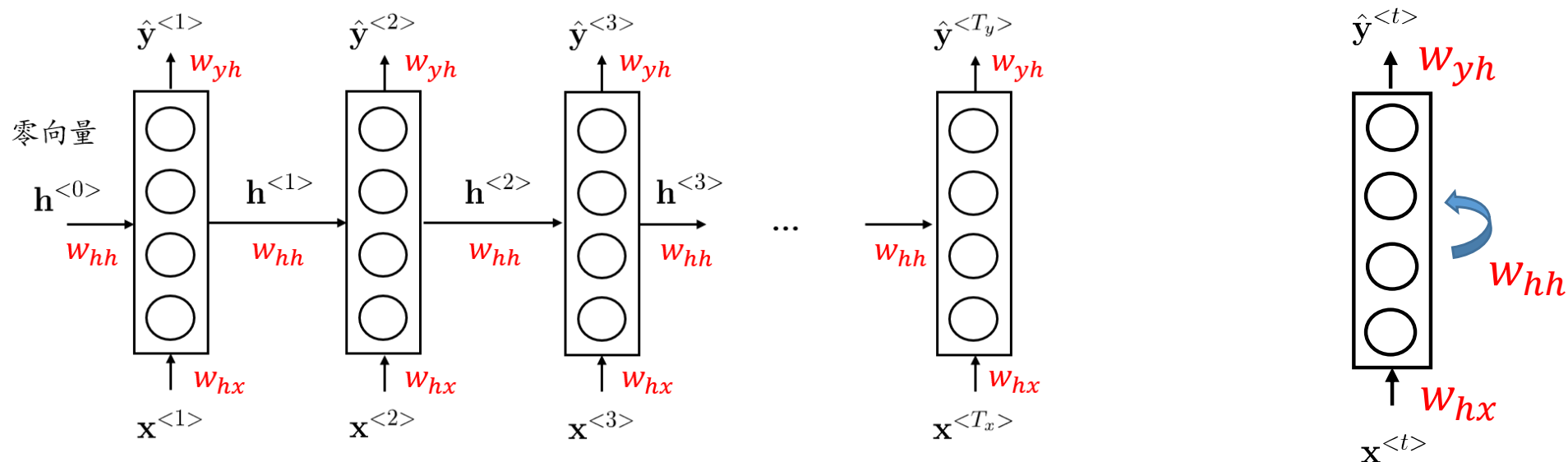
01 长短时记忆网络 (LSTM)

02 门控循环单元(GRU)

03 循环神经网络应用

04 小结和资源





$$h^{<0>} = 0 \quad \begin{cases} h^{<1>} = g(W_{hh}h^{<0>} + W_{hx}x^{<1>} + b_h) \\ \hat{y}^{<1>} = g_2(W_{yh}h^{<1>} + b_y) \end{cases}$$

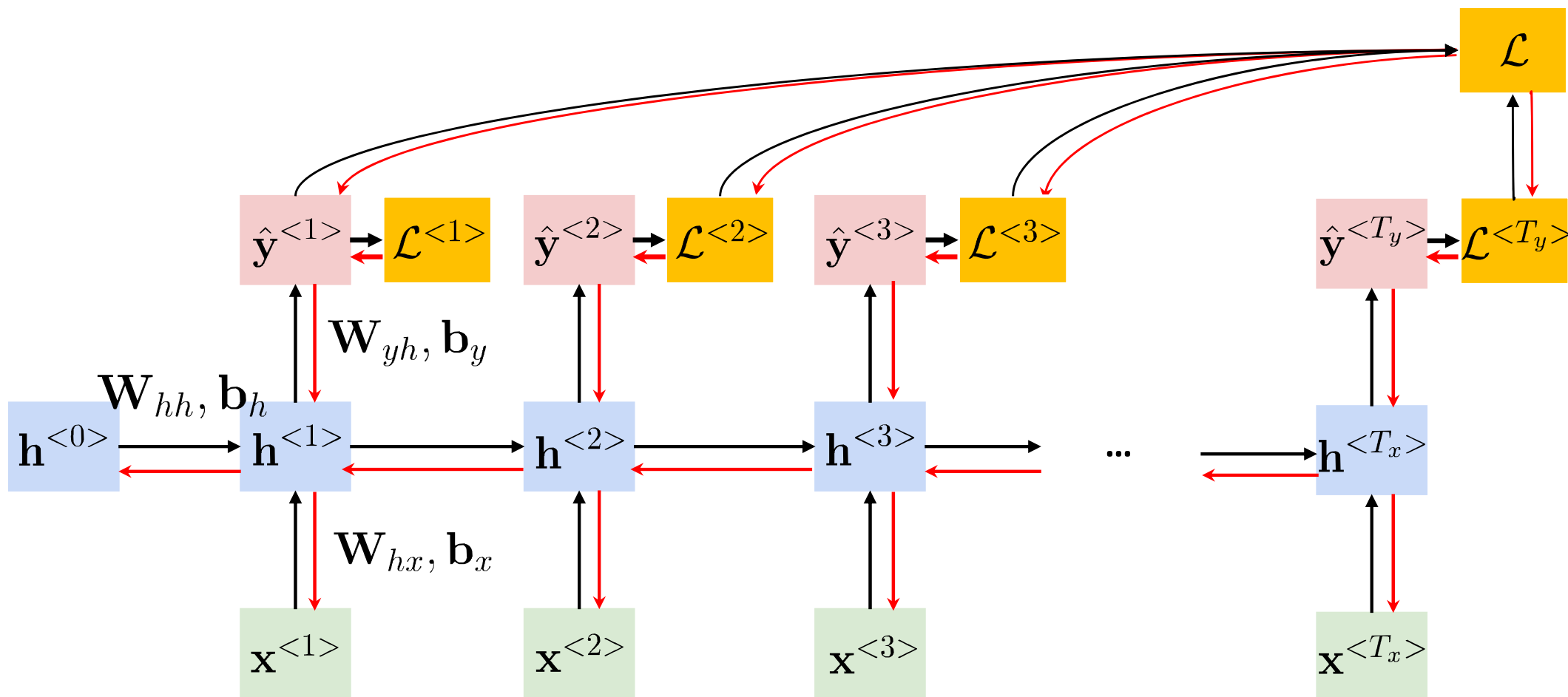
$$\begin{cases} h^{<t>} = g(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b_h) \\ \hat{y}^{<t>} = g_2(W_{yh}h^{<t>} + b_y) \end{cases}$$

$$\begin{cases} h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h) \\ \hat{y}^{<t>} = g(W_y h^{<t>} + b_y) \end{cases}$$

激活函数: tanh

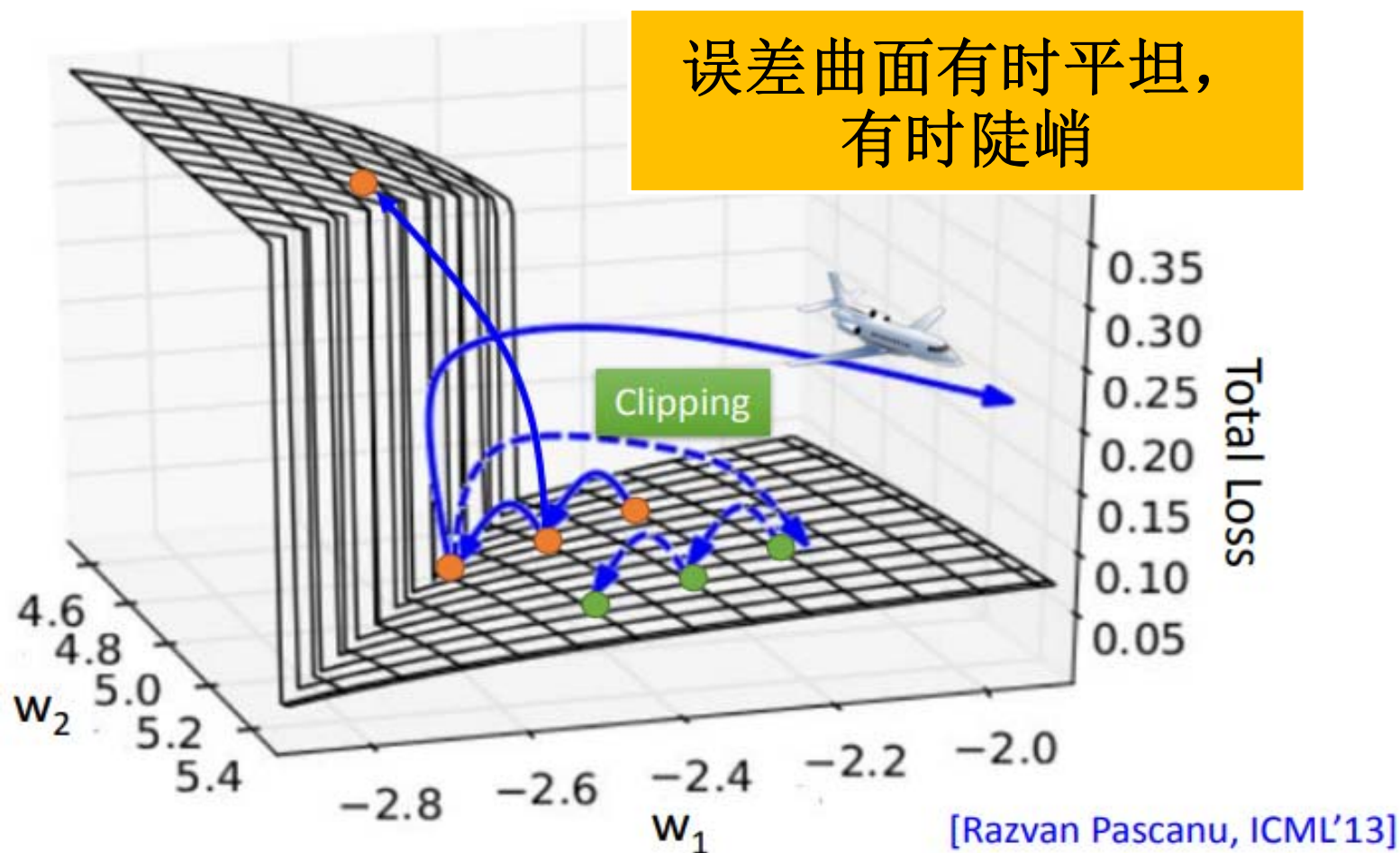
激活函数: sigmoid (二分类)
softmax (多分类)

- BPTT (Back-propagation through time): 随时间反向传播



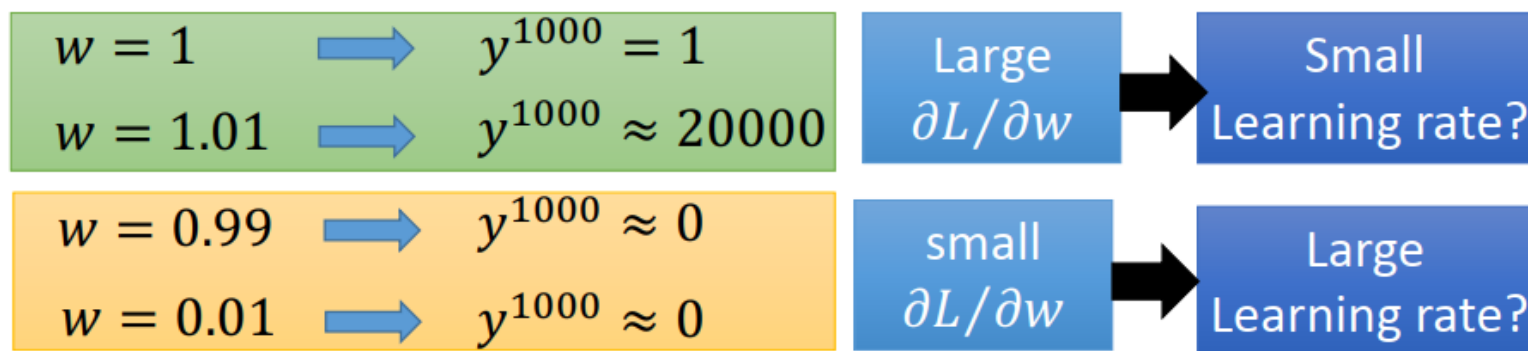
- BPTT (Back-propagation through time): 随时间反向传播

□ 训练RNN通常比较困难

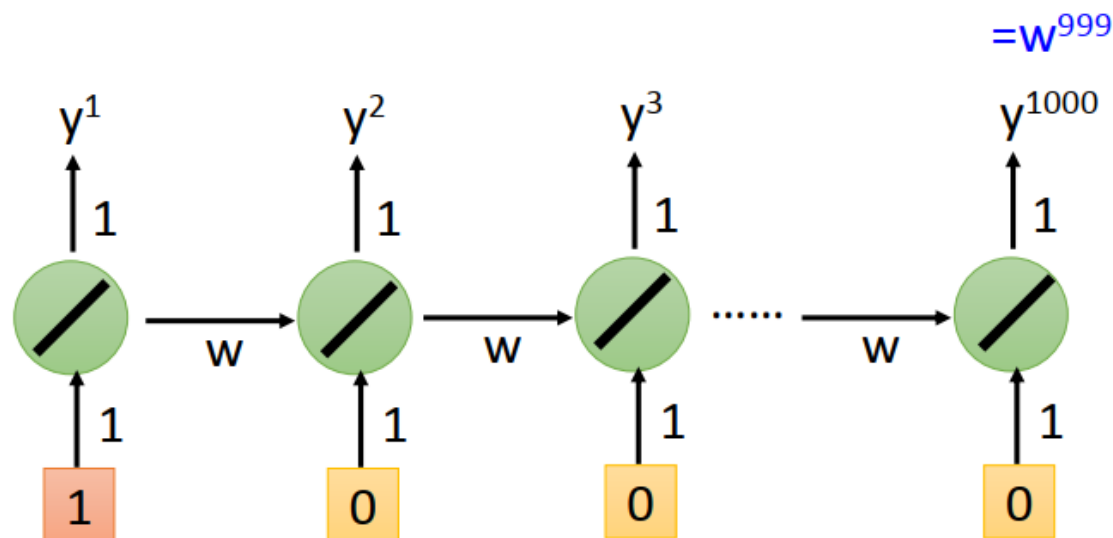


- BPTT (Back-propagation through time): 随时间反向传播

□ 训练RNN通常比较困难



Toy Example



1 循环神经网络训练



- 对 $\mathbf{W}_{hh}$ 求偏导:

$$\frac{\partial \mathcal{L}^{<t>}}{\partial \mathbf{W}_{hh}} = \sum_{k=0}^t \frac{\partial \mathcal{L}^{<t>}}{\partial \hat{y}^{<t>}} \cdot \frac{\hat{y}^{<t>}}{\partial \mathbf{h}^{<t>}} \cdot \left(\prod_{j=k+1}^t \frac{\partial \mathbf{h}^{<j>}}{\partial \mathbf{h}^{<j-1>}} \right) \cdot \frac{\partial \mathbf{h}^{<k>}}{\partial \mathbf{W}_{hh}}$$

$$\prod_{j=k+1}^t (\tanh)' \cdot \mathbf{W}_{hh}$$

- 对 $\mathbf{W}_{hx}$ 求偏导:

$$\frac{\partial \mathcal{L}^{<t>}}{\partial \mathbf{W}_{hx}} = \sum_{k=0}^t \frac{\partial \mathcal{L}^{<t>}}{\partial \hat{y}^{<t>}} \cdot \frac{\hat{y}^{<t>}}{\partial \mathbf{h}^{<t>}} \cdot \left(\prod_{j=k+1}^t \frac{\partial \mathbf{h}^{<j>}}{\partial \mathbf{h}^{<j-1>}} \right) \cdot \frac{\partial \mathbf{h}^{<k>}}{\partial \mathbf{W}_{hx}}$$

$$\prod_{j=k+1}^t (\text{sigmoid})' \cdot \mathbf{W}_{hh}$$

连乘会导致激活函数导数的连乘，进而会导致“梯度消失”和“梯度爆炸”现象的发生

$$\mathbf{h}^{<t>} = g(\mathbf{W}_{hh}\mathbf{h}^{<t-1>} + \mathbf{W}_{hx}\mathbf{x}^{<t>} + \mathbf{b}_h)$$

$$\hat{y}^{<t>} = g_2(\mathbf{W}_{yh}\mathbf{h}^{<t>} + \mathbf{b}_y)$$

$$\mathcal{L}(\hat{\mathbf{y}}^{<t>}, \mathbf{y}^{<t>}) = - \sum_i y_i^{<t>} \log \hat{y}_i^{<t>}$$

$$\mathcal{L} = \sum_t \mathcal{L}^{<t>}(\hat{\mathbf{y}}^{<t>}, \mathbf{y}^{<t>})$$

01

PART ONE

长短期记忆网络

Long-Short Term Memory

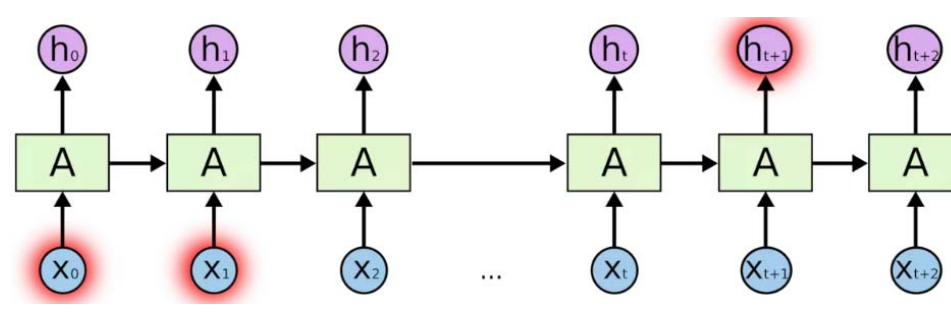
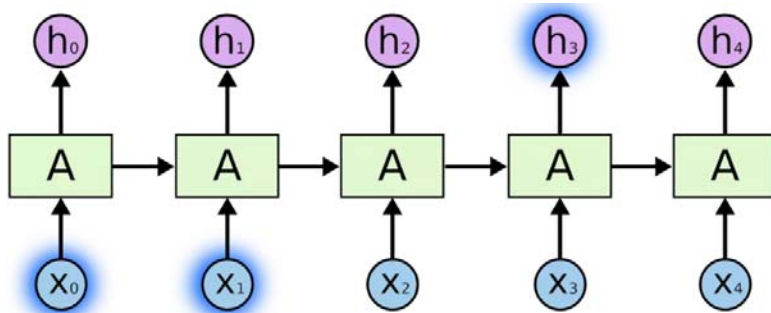
Andrew Ng, Recurrent Neural Network

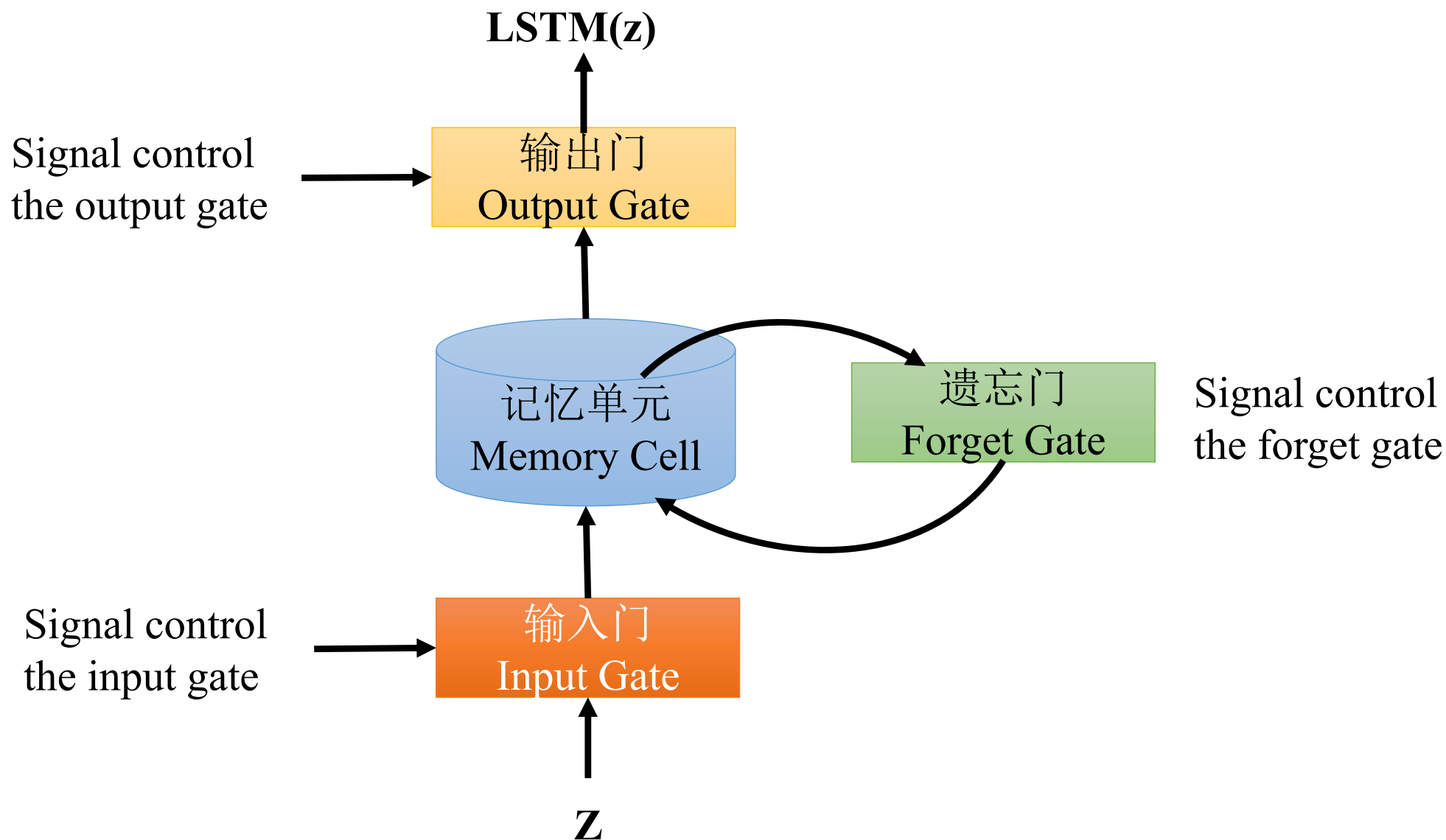
<https://www.bilibili.com/video/av88010803?p=1>

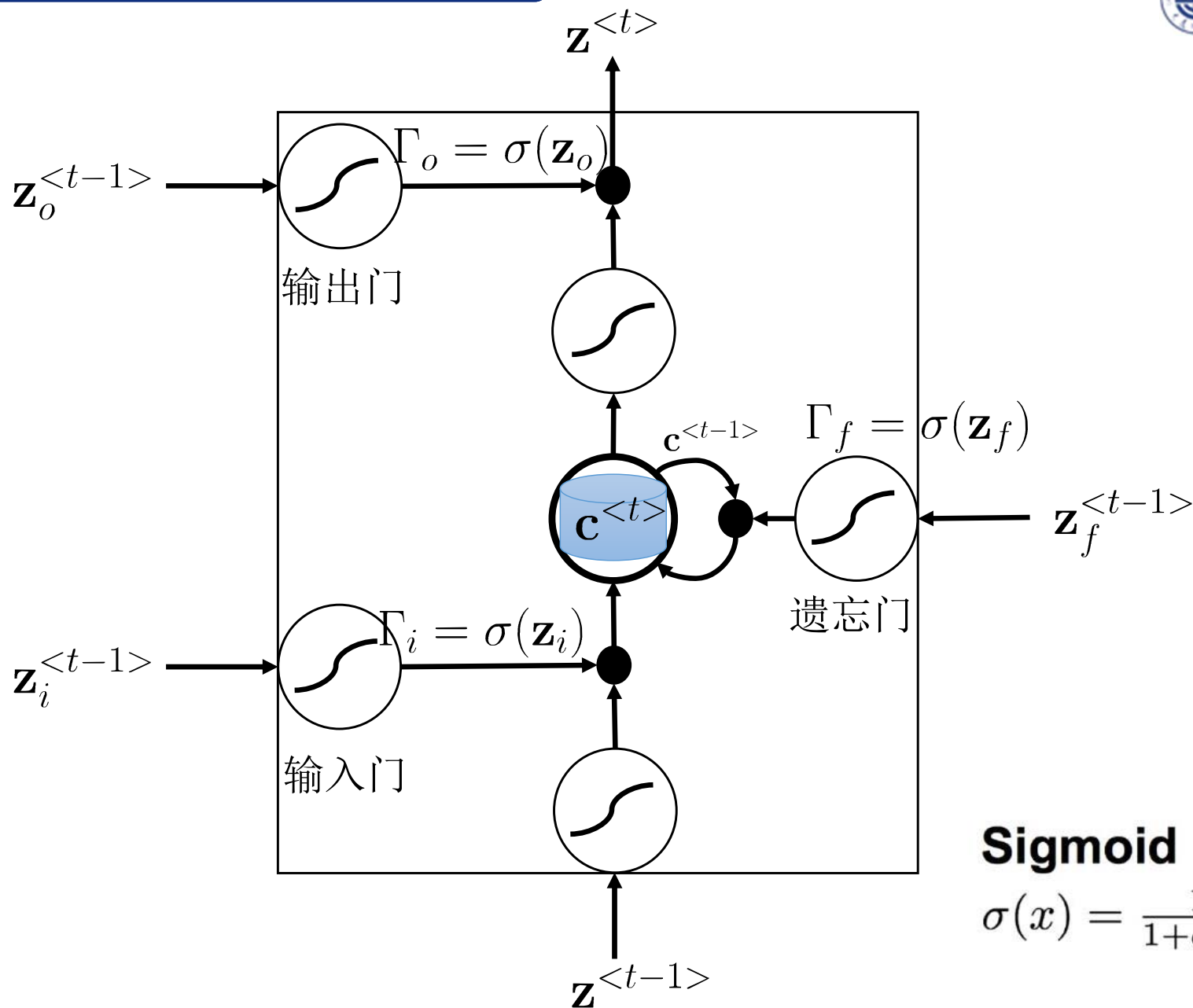
The **cat**, which already ate, **was** full.

The **cats**, which already ate, **were** full.

- 序列短时，相关的信息和预测的词位置之间的间隔小，**RNN** 可以学会使用先前的信息。
- 序列长时，相关信息和当前预测位置之间的间隔大，**RNN** 会丧失学习到连接如此远的信息的能力。
- 另外，序列过长时，在参数优化的过程中会发现“梯度爆炸”或“梯度消失”的情况。

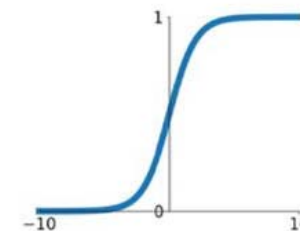


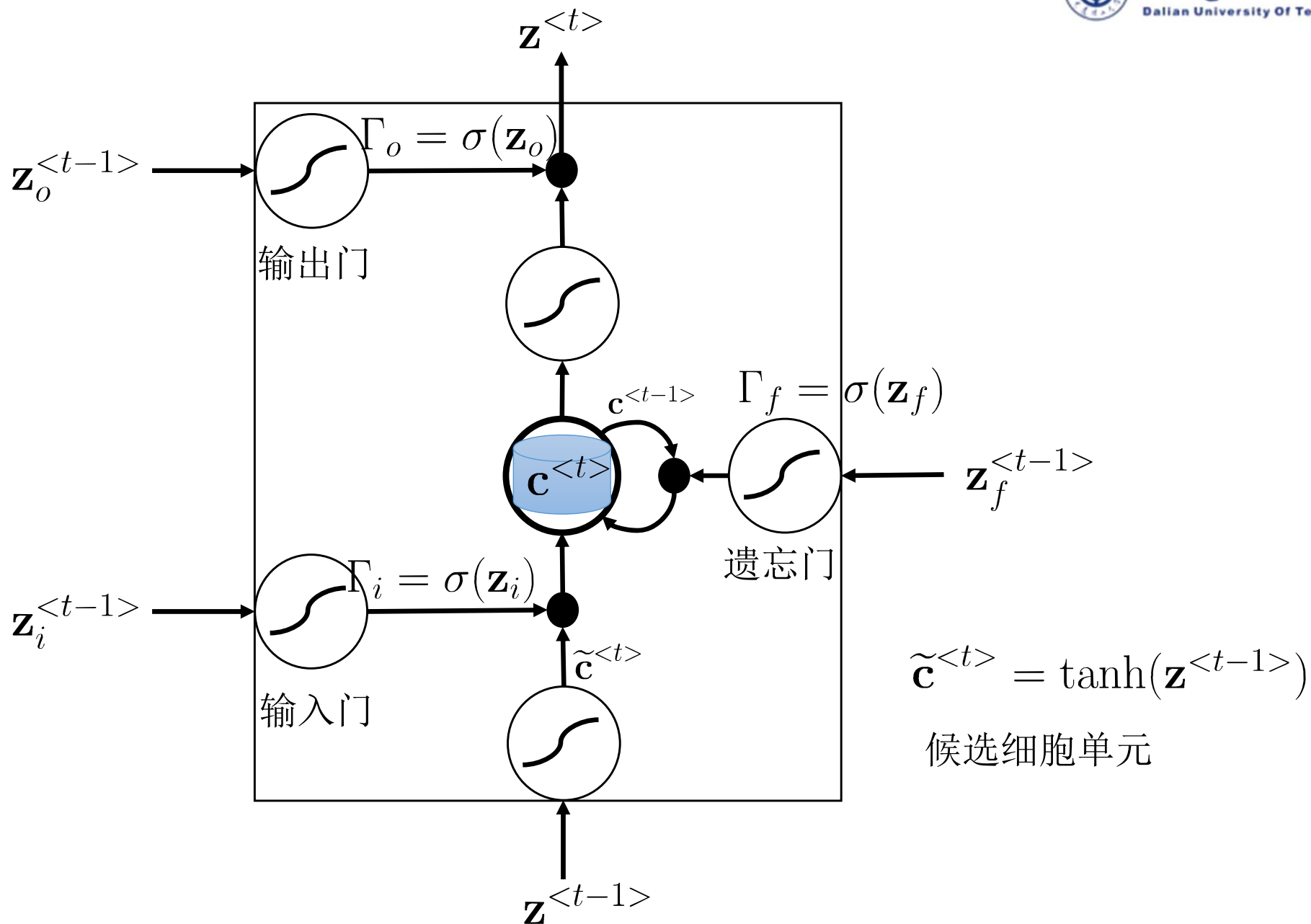


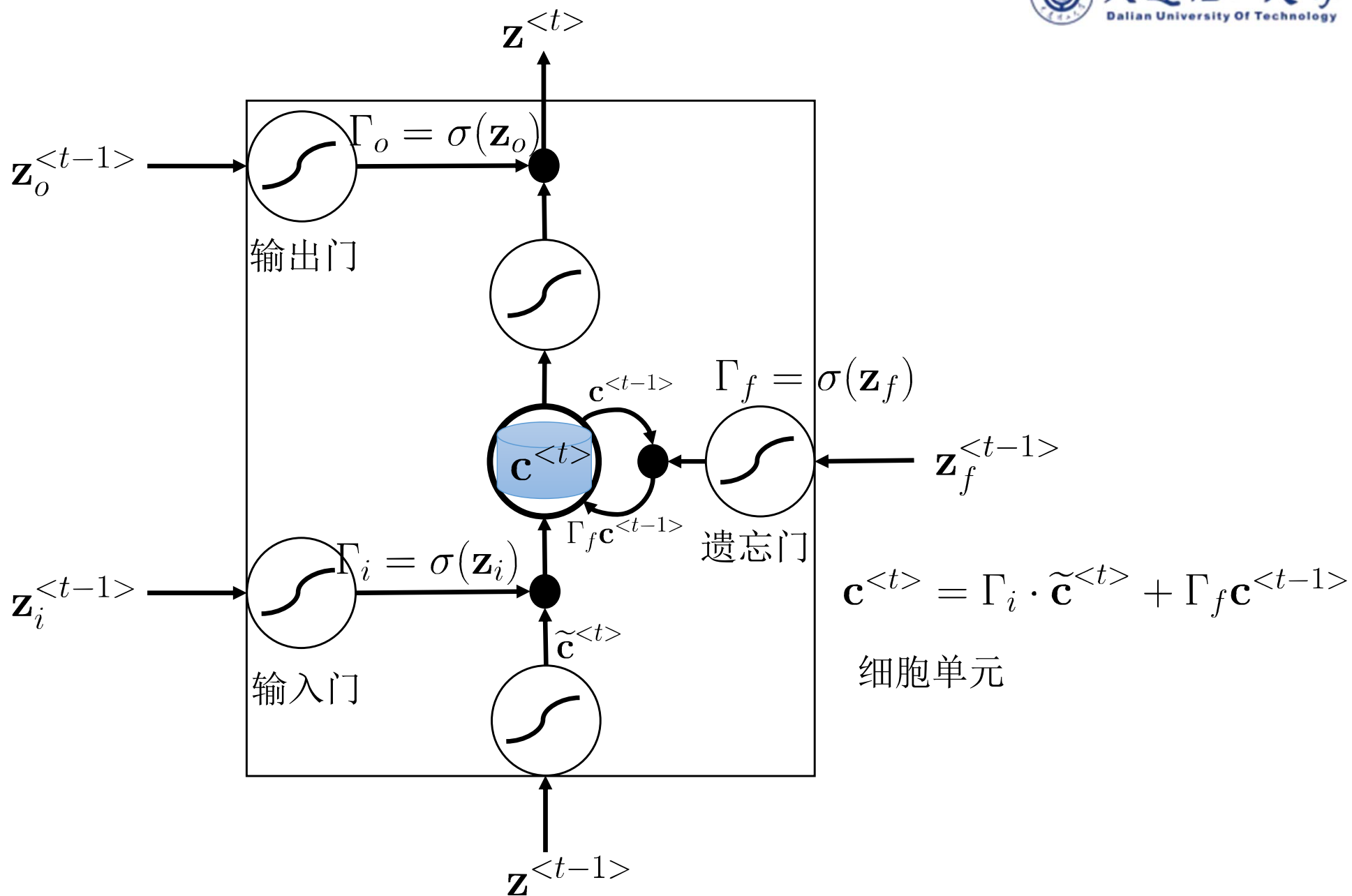


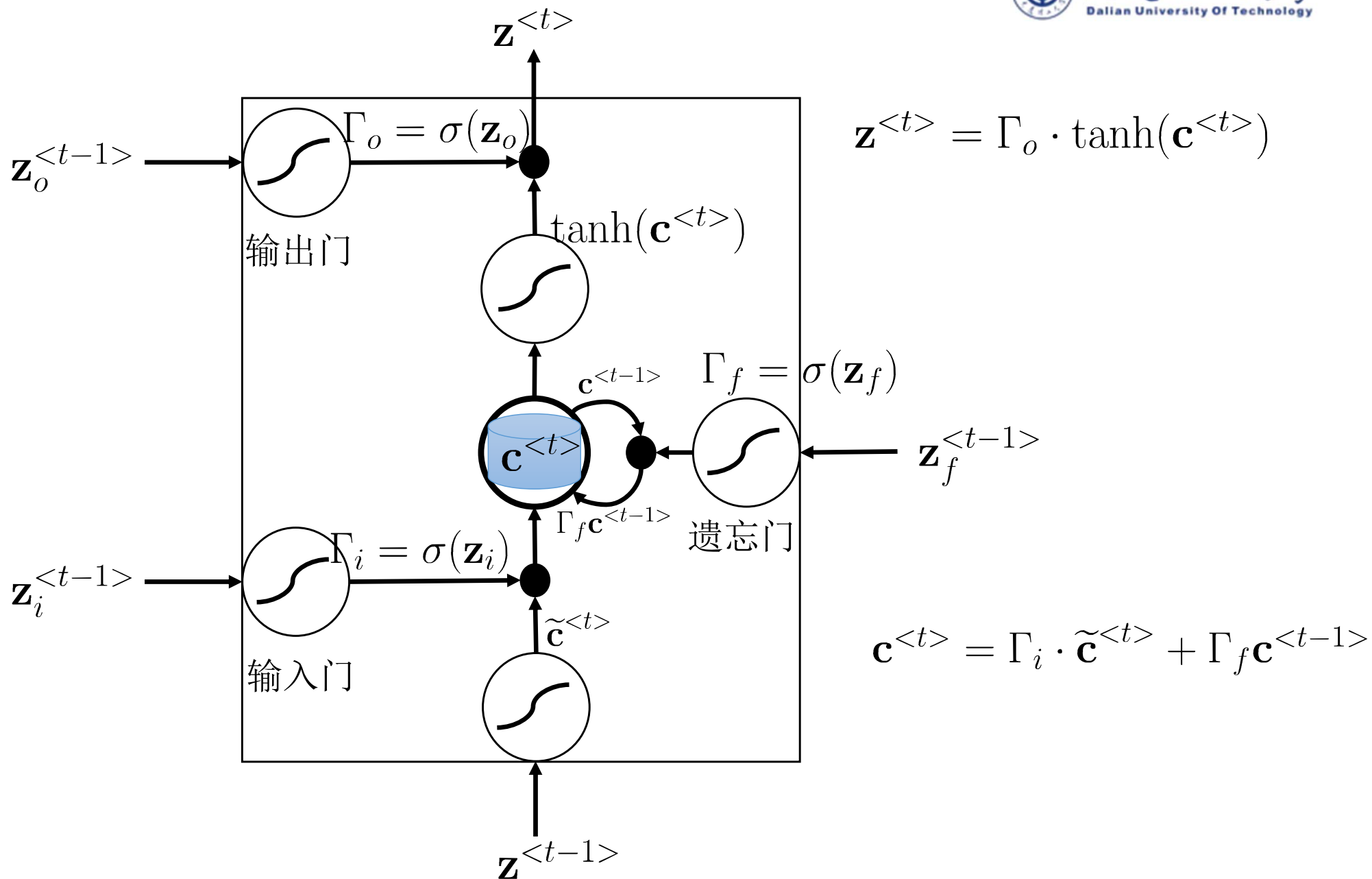
Sigmoid

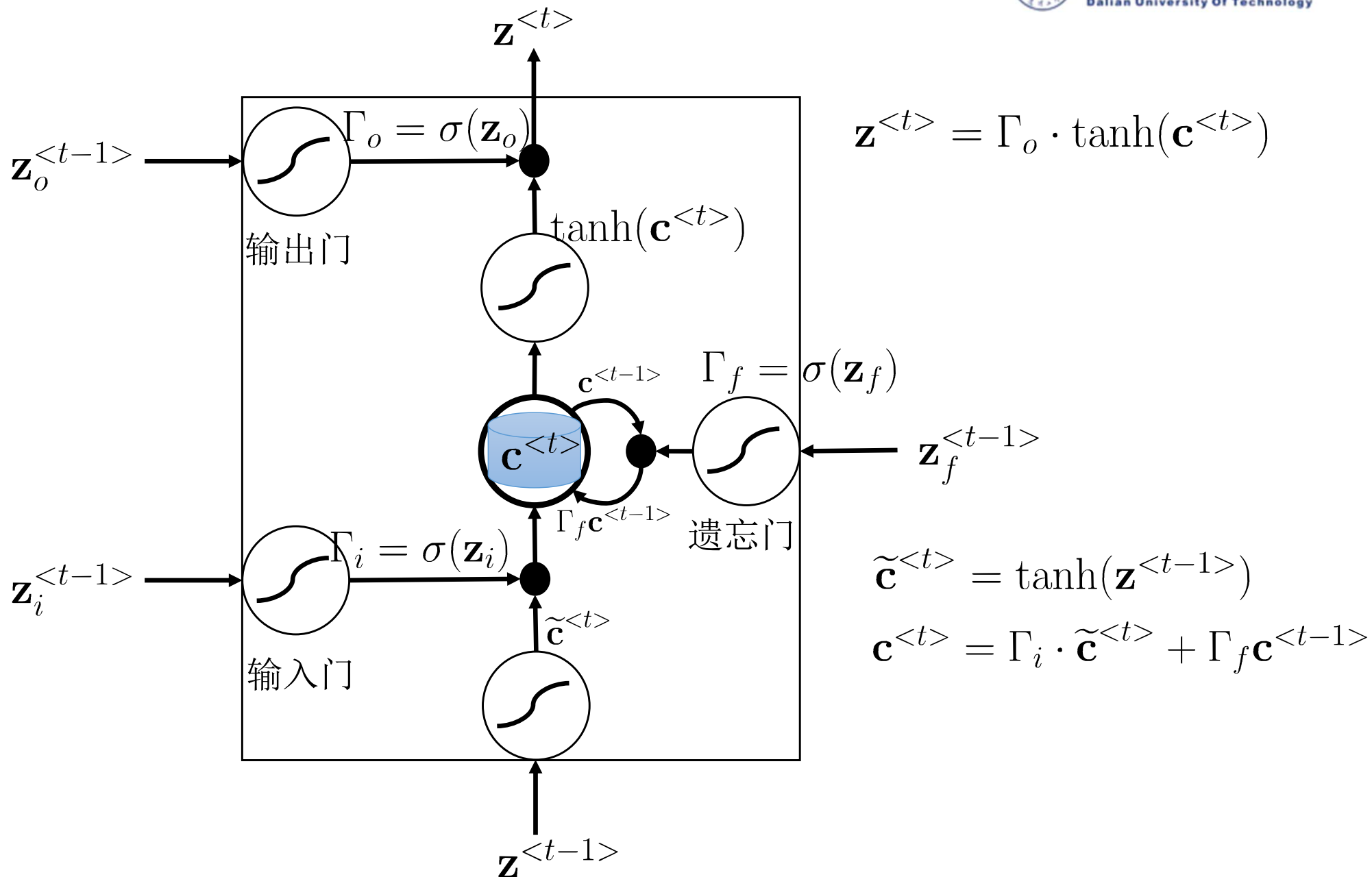
$$\sigma(x) = \frac{1}{1+e^{-x}}$$





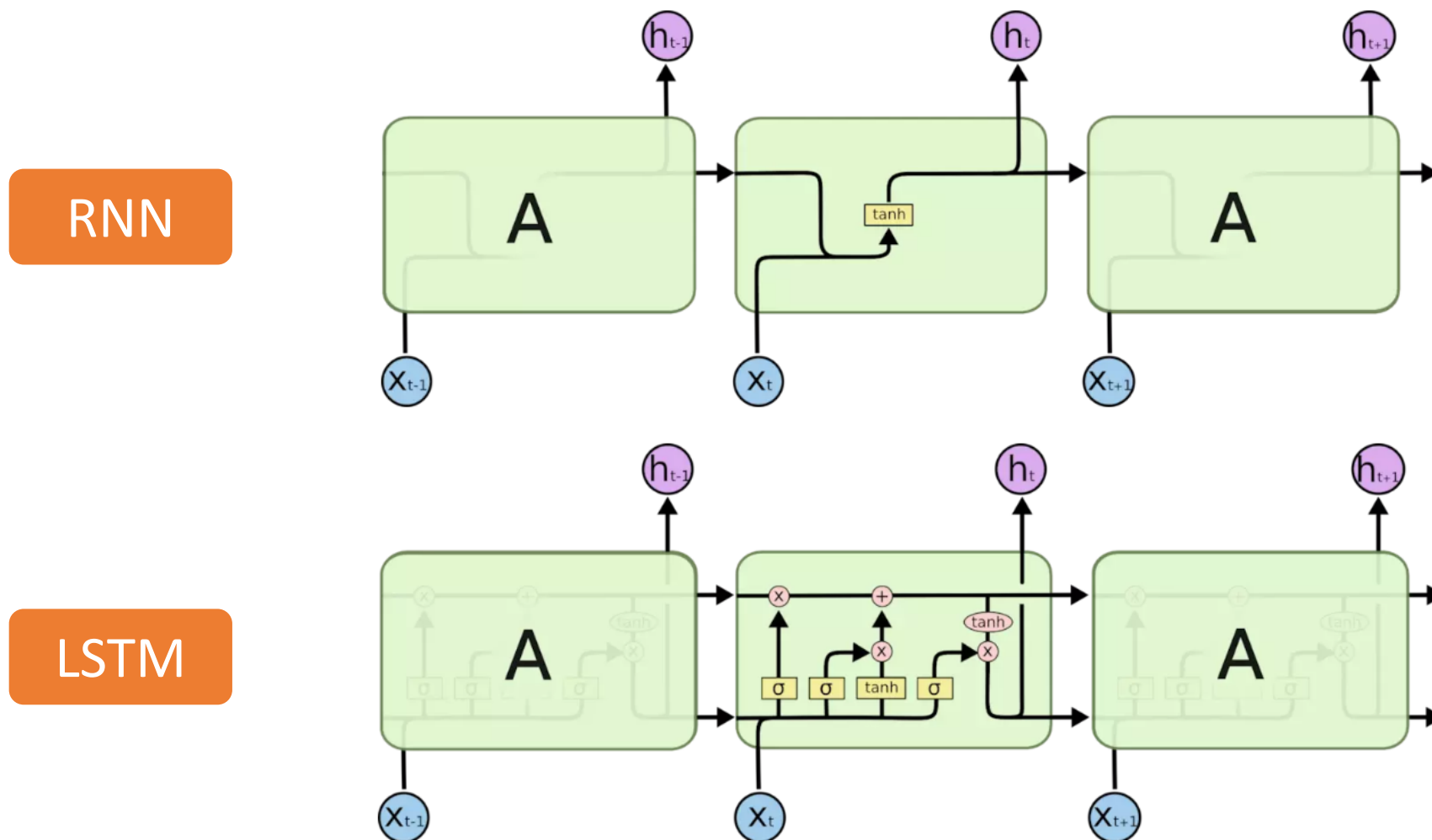








- LSTM (长短期记忆网络, Long-Short Term Memory Network): 是一种 RNN 特殊的类型, 可以学习长期依赖信息。

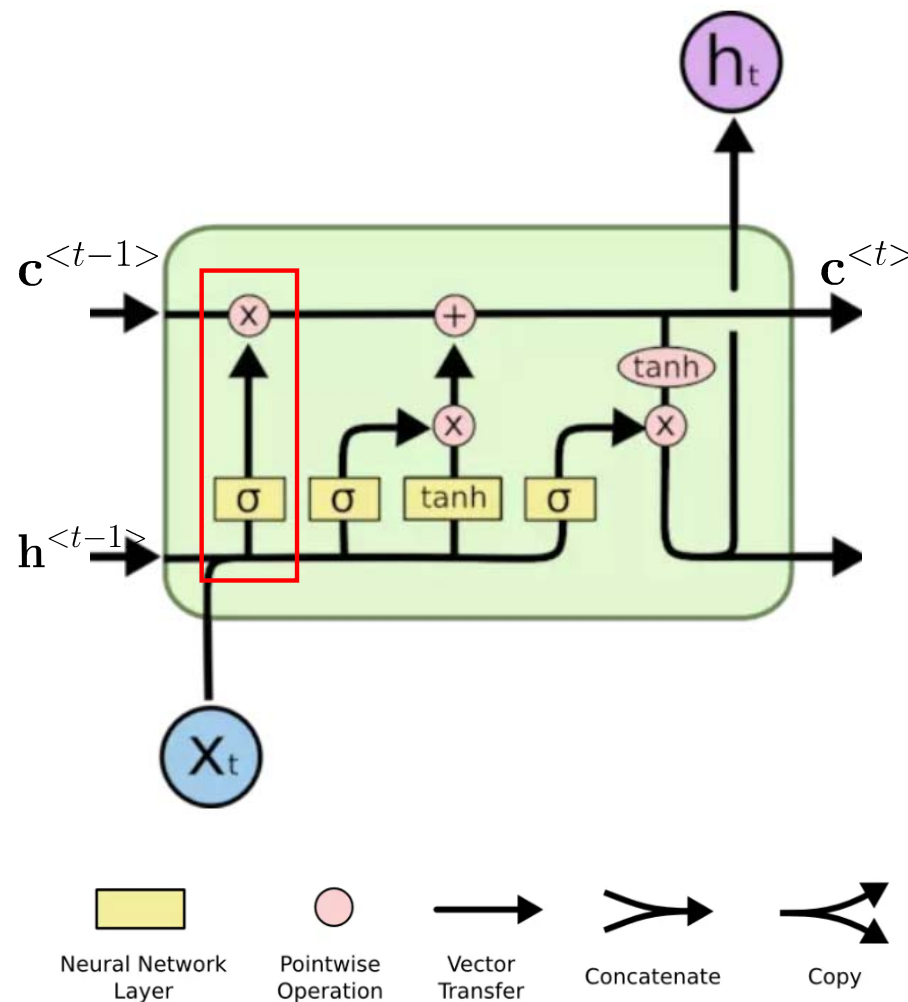


$C^{<t>}$: 记忆单元, 也称细胞状态, 提供记忆能力, 记住以前的信息。

细胞状态类似于传送带。直接在整个链上运行, 只有一些少量的线性交互。信息在上面流传保持不变会很容易。

门结构: 让信息选择式通过的方法, 向细胞状态去除或者增加信息。

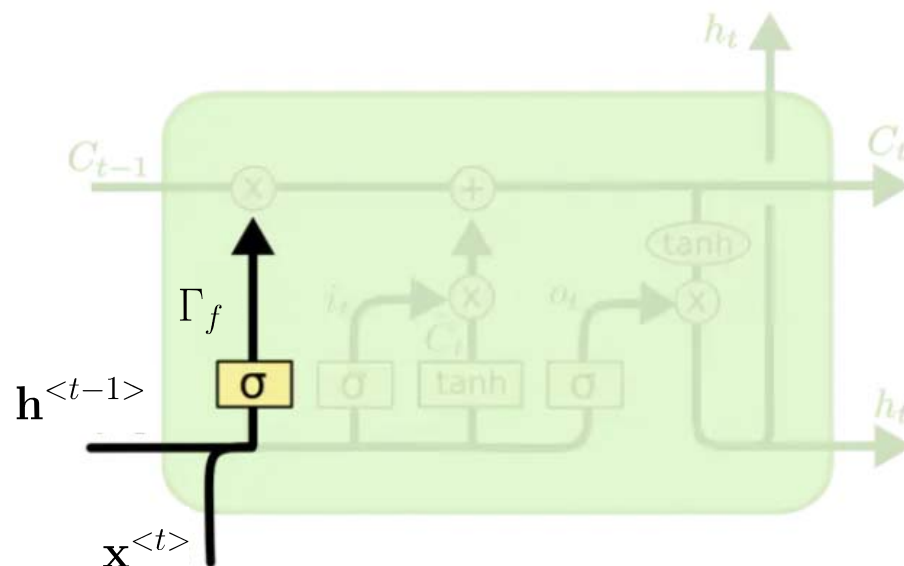
Sigmoid层输出0到1间的数值, 控制信息的通过量。LSTM 拥有三个门, 来保护和控制细胞状态。



遗忘门 Γ_f : 决定从细胞状态中丢弃什么信息。

$$\Gamma_f = \sigma(\mathbf{W}_f[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_f)$$

遗忘门根据 $\mathbf{h}^{<t-1>}$, $\mathbf{x}^{<t>}$ 输出0到1间的数值给细胞状态 $\mathbf{c}^{<t-1>}$ 。1表示完全保留，0表示完全舍弃。



The **cat**, which already ate, **was** full.

The **cats**, which already ate, **were** full.

当前细胞状态可能包含当前主语的单复数(cat/cats), 因此正确的系动词可以被选择出来。当我们看到新的主语, 我们希望忘记旧的主语。

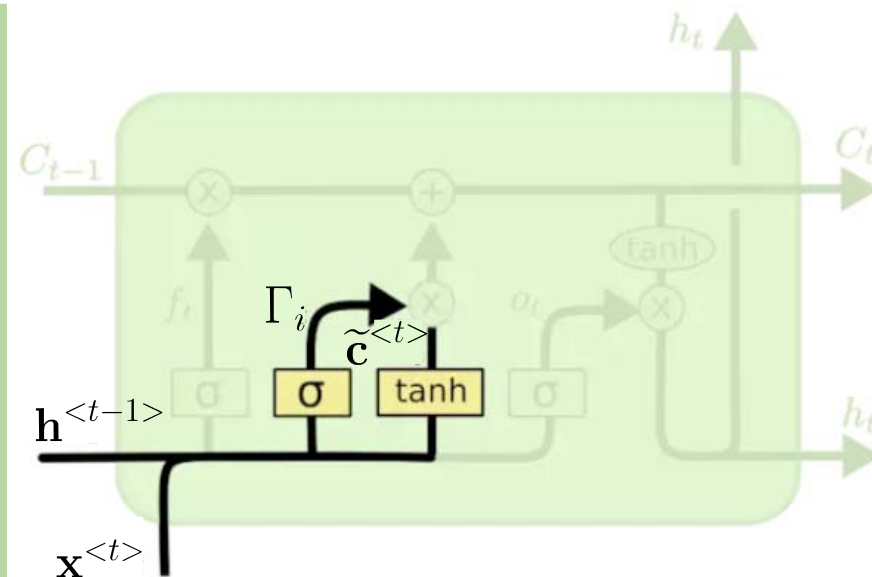
记忆门（输入门）：确定什么样的新信息被存放在细胞状态中

- Sigmoid层：决定将要更新什么值

$$\Gamma_i = \sigma(\mathbf{W}_i[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i)$$

- Tanh层：创建一个新的候选细胞单元 $\tilde{\mathbf{c}}^{<t>}$ 会被加入到状态中

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_c[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c)$$

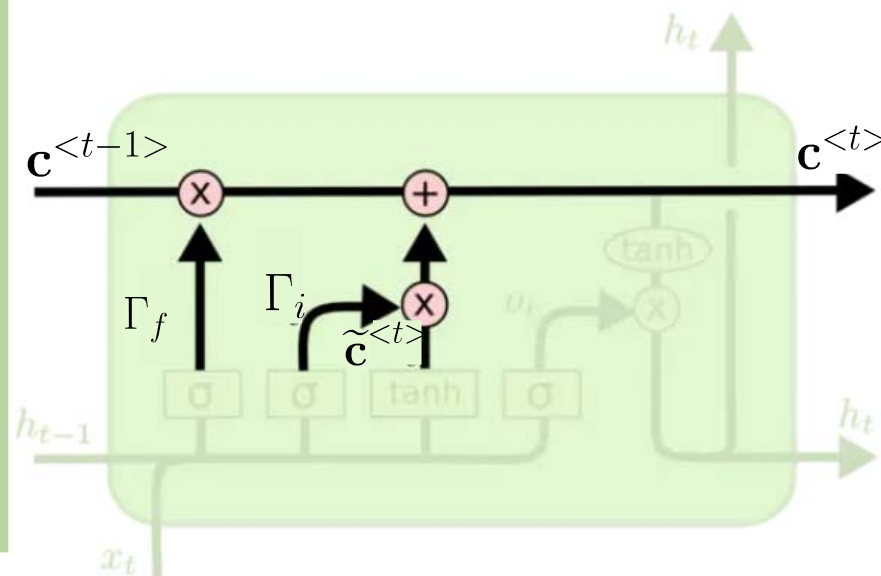


例如在语言模型中，如希望增加新的主语的单复数到细胞状态中，来替代旧的需要忘记的主语。

更新旧细胞状态，即 $\mathbf{c}^{<t-1>}$ 更新为 $\mathbf{c}^{<t>}$

- 遗忘门与旧状态相乘，遗忘掉确定丢弃的信息
- 记忆门与候选细胞单元相乘

$$\mathbf{c}^{<t>} = \Gamma_f * \mathbf{c}^{<t-1>} + \Gamma_i * \tilde{\mathbf{c}}^{<t>}$$



语言模型中，根据前面确定的目标，丢弃旧单词的单复数信息并添加新的信息。

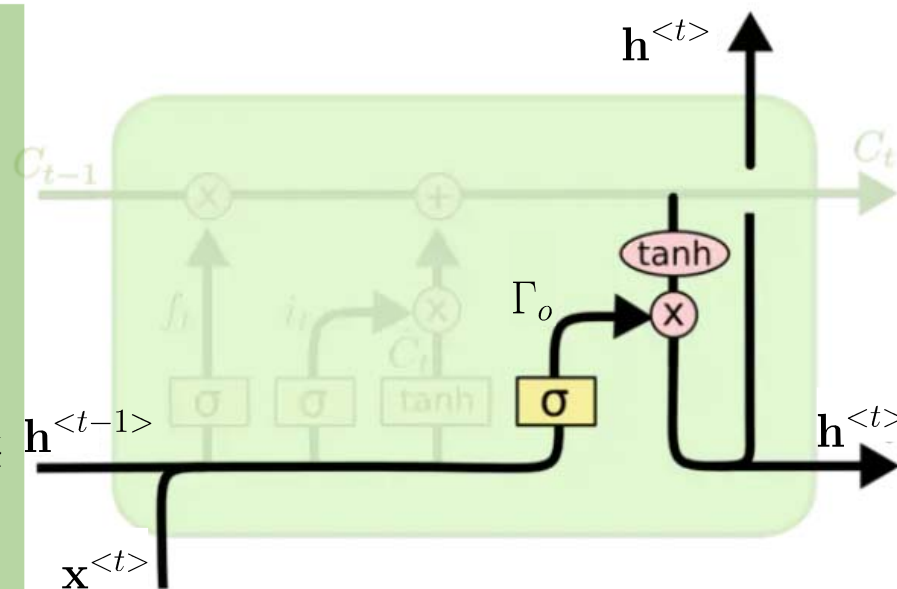
输出门：需要确定输出什么值

- Sigmoid层：确定输出细胞状态的哪个部分

$$\Gamma_o = \sigma(\mathbf{W}_o[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_o)$$

- 把细胞状态通过tanh进行处理，与输出门相乘

$$\mathbf{h}^{<t>} = \Gamma_o * \tanh(\mathbf{c}^{<t>})$$



$$\Gamma_f = \sigma(\mathbf{W}_f[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_f)$$

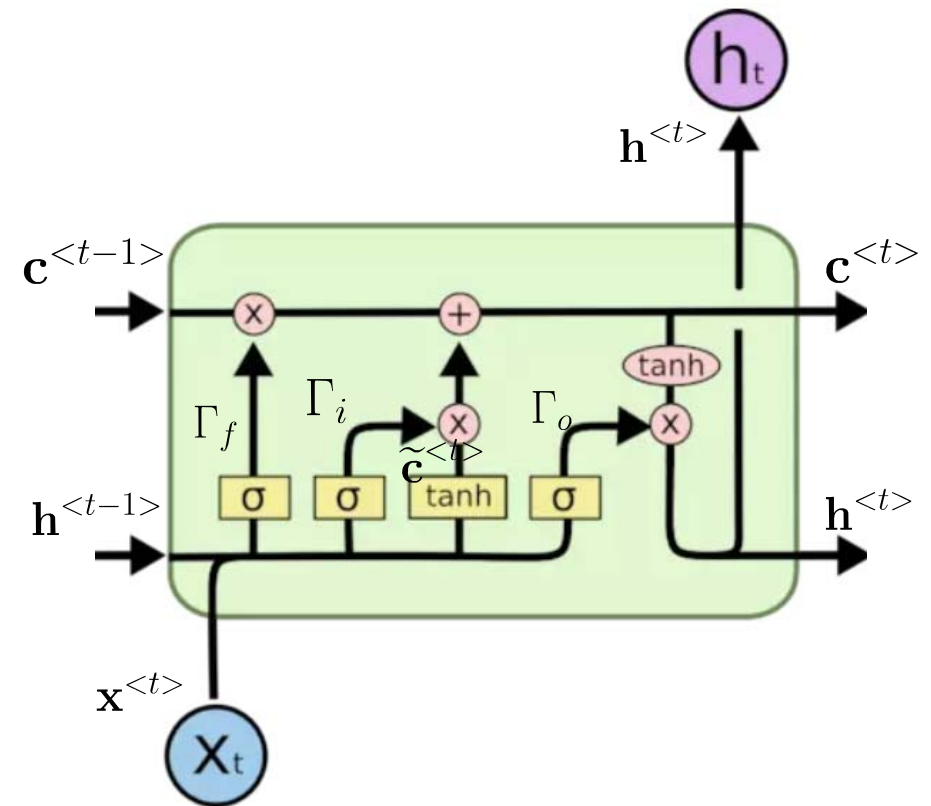
$$\Gamma_i = \sigma(\mathbf{W}_i[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i)$$

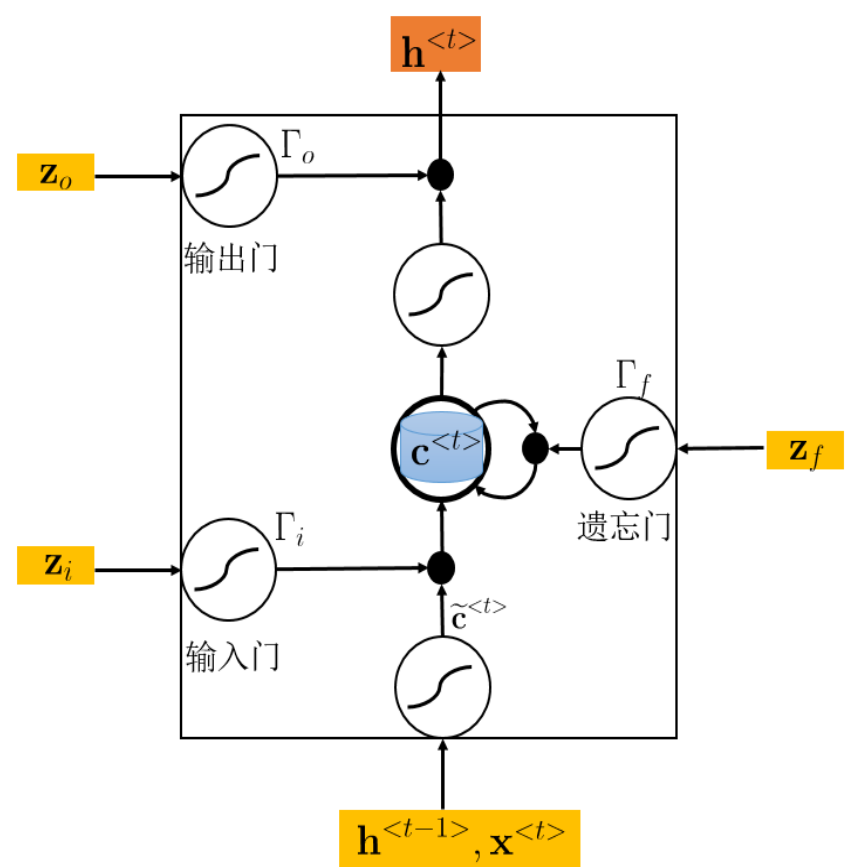
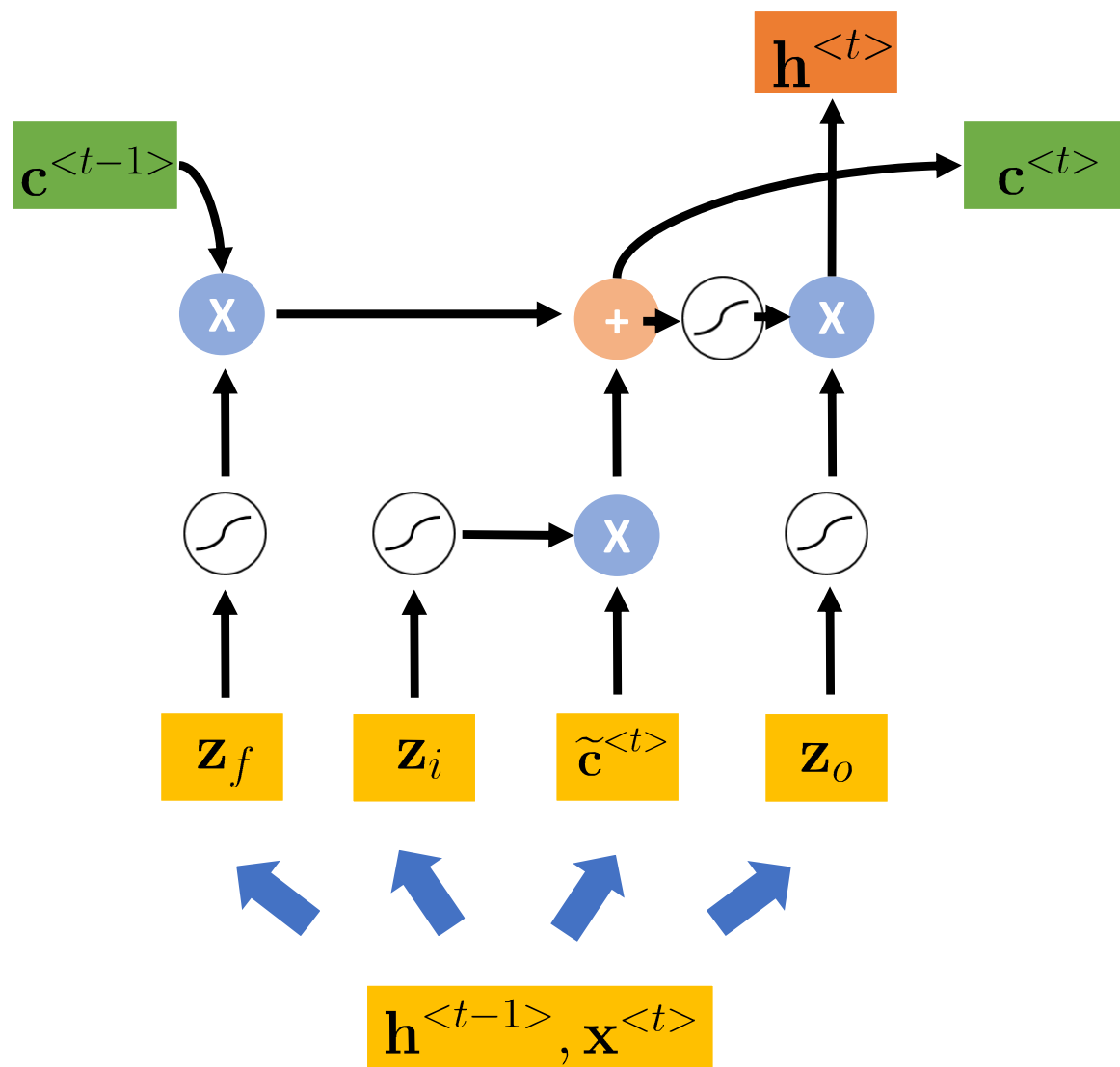
$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_c[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c)$$

$$\mathbf{c}^{<t>} = \Gamma_f * \mathbf{c}^{<t-1>} + \Gamma_i * \tilde{\mathbf{c}}^{<t>}$$

$$\Gamma_o = \sigma(\mathbf{W}_o[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_o)$$

$$\mathbf{h}^{<t>} = \Gamma_o * \tanh(\mathbf{c}^{<t>})$$





$$\Gamma_f = \sigma(\mathbf{W}_f[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_f)$$

$$\Gamma_i = \sigma(\mathbf{W}_i[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i)$$

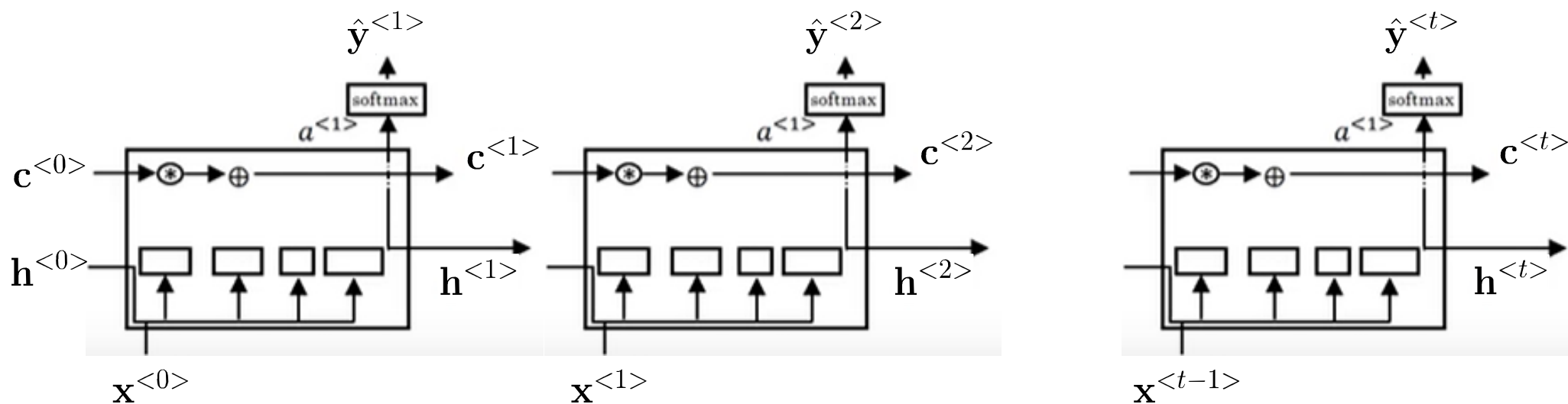
$$\tilde{c}^{<t>} = \tanh(\mathbf{W}_c[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c)$$

$$c^{<t>} = \Gamma_f * c^{<t-1>} + \Gamma_i * \tilde{c}^{<t>}$$

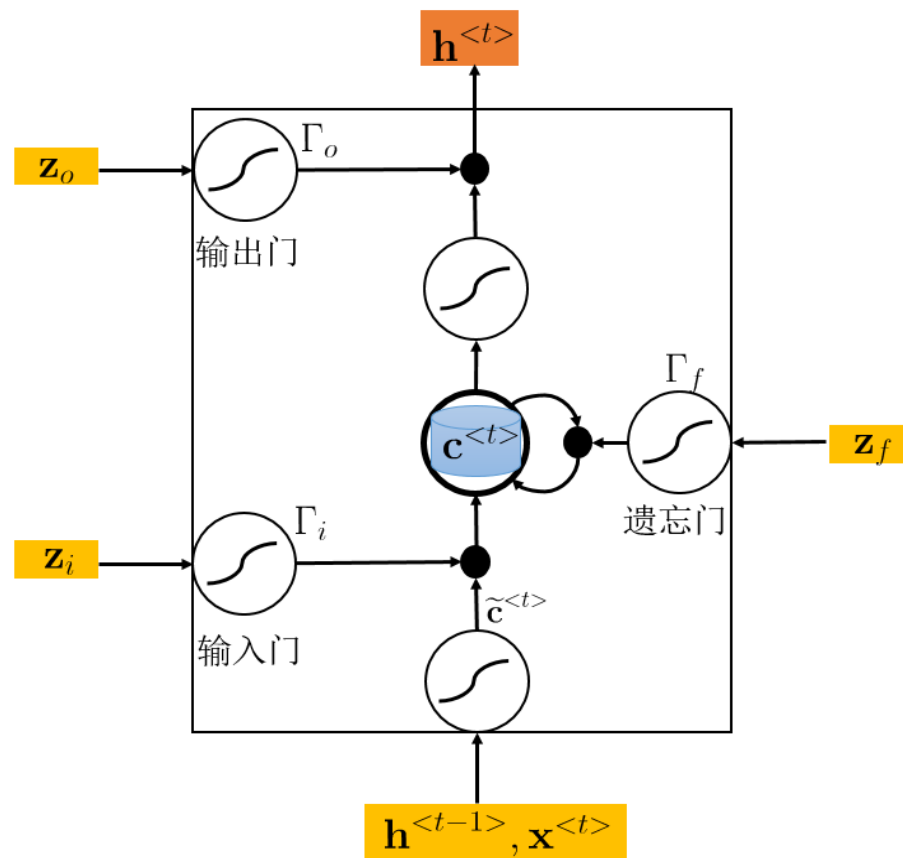
$$\Gamma_o = \sigma(\mathbf{W}_o[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_o)$$

$$h^{<t>} = \Gamma_o * \tanh(c^{<t>})$$

■ 长短时记忆网络



- 长短时记忆网络
- 能够处理梯度消失的情况
 - Memory + input
 - 只要遗忘门开启，之前记忆的影响一直在



PART TWO

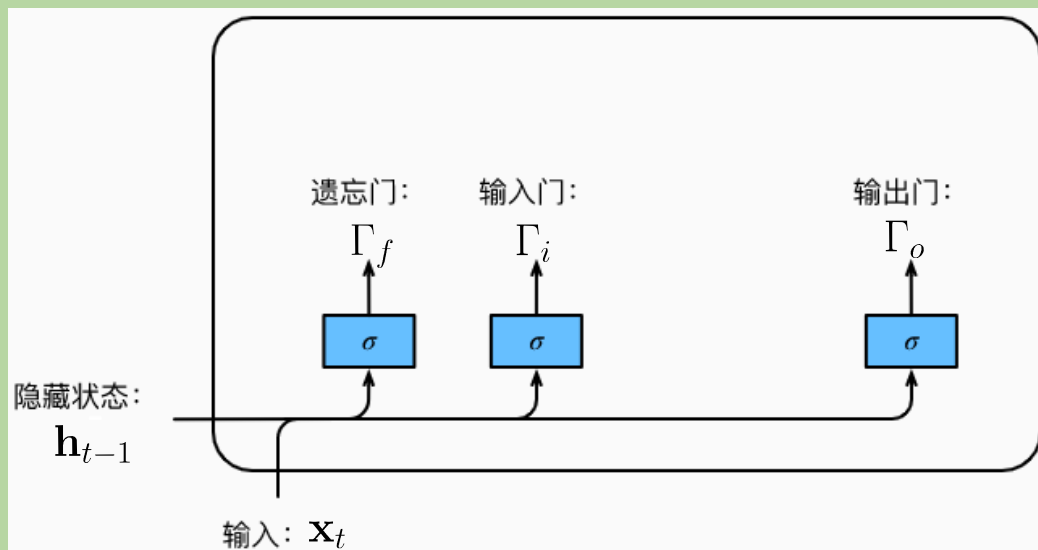
门控循环单元

Gated Recurrent Unit

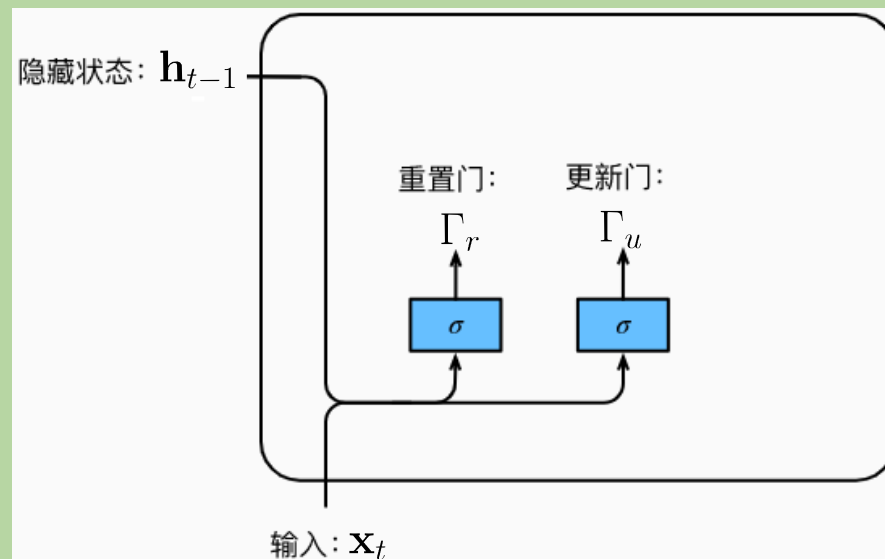


- GRU (门控循环单元, Gated Recurrent Unit): 是一种 RNN 特殊的类型, 可以学习长期依赖信息。

LSTM



GRU



- 重置门：确定是否丢弃上一时间步的隐藏状态

$$\Gamma_r = \sigma(\mathbf{W}_r[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_r)$$

- 更新门：确定是否更新上一时间步的隐藏状态

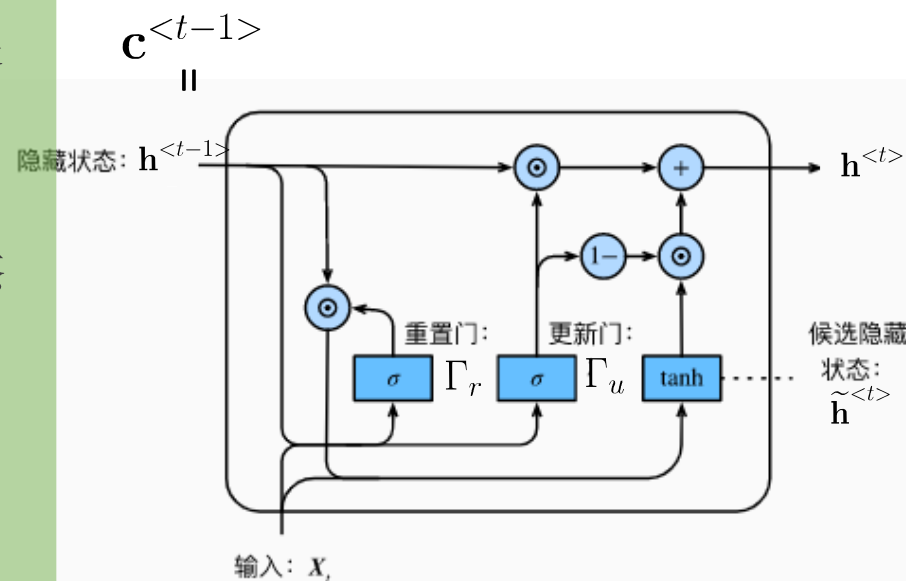
$$\Gamma_u = \sigma(\mathbf{W}_u[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_u)$$

- 候选隐藏状态：

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_h[\Gamma_r \mathbf{c}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_h)$$

- 隐藏状态：

$$\mathbf{h}^{<t>} = \Gamma_u * \mathbf{h}^{<t-1>} + (1 - \Gamma_u) * \tilde{\mathbf{c}}^{<t>}$$



1

GRU

LSTM

$$\Gamma_f = \sigma(\mathbf{W}_f[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_f)$$

$$\Gamma_i = \sigma(\mathbf{W}_i[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_c[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_c)$$

$$\mathbf{c}^{<t>} = \Gamma_f * \mathbf{c}^{<t-1>} + \Gamma_i * \tilde{\mathbf{c}}^{<t>}$$

$$\Gamma_o = \sigma(\mathbf{W}_o[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_o)$$

$$\mathbf{h}^{<t>} = \Gamma_o * \tanh(\mathbf{c}^{<t>})$$

GRU

$$\Gamma_r = \sigma(\mathbf{W}_r[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_r)$$

$$\Gamma_u = \sigma(\mathbf{W}_u[\mathbf{h}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_u)$$

$$\tilde{\mathbf{c}}^{<t>} = \tanh(\mathbf{W}_h[\Gamma_r \mathbf{c}^{<t-1>}, \mathbf{x}^{<t>}] + \mathbf{b}_h)$$

$$\mathbf{h}^{<t>} = \Gamma_u * \mathbf{h}^{<t-1>} + (1 - \Gamma_u) * \tilde{\mathbf{c}}^{<t>}$$

PART THREE

循环神经网络应用

RNN Applications

- 词嵌入 (Word embedding) 是自然语言处理 (NLP) 中语言模型与表征学习技术的统称。概念上而言, 它是指把一个维数为所有词的数量的高维空间嵌入到一个维数低得多的连续向量空间中, 每个单词或词组被映射为实数域上的向量。



- 文本表示
 - 整数编码
 - One-hot编码
 - 词嵌入(word embedding)

■ 整数编码

整数编码



1



2



3



4

■ 缺点：

- 无法表达词语之间的关系
- 对于模型解释而言，整数编码可能具有挑战性

■ One-hot编码

one-hot 编码

[, , , ]

词汇表
Vocabulary

 = [1, 0, 0, 0]
 = [0, 1, 0, 0]
 = [0, 0, 1, 0]
 = [0, 0, 0, 1]

■ 缺点:

- ❑ 无法表达词语之间的关系，相关词汇间的泛化能力不强
- ❑ 对于这种过于稀疏的向量，计算和存储效率都不高

I want a glass of orange _____.

I want a glass of apple _____.

■ 词嵌入(word embedding): 特征化表征

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97
⋮	⋮	⋮	⋮	⋮	⋮	⋮

300*1维

e_{5391} e_{9853} e_{4919}

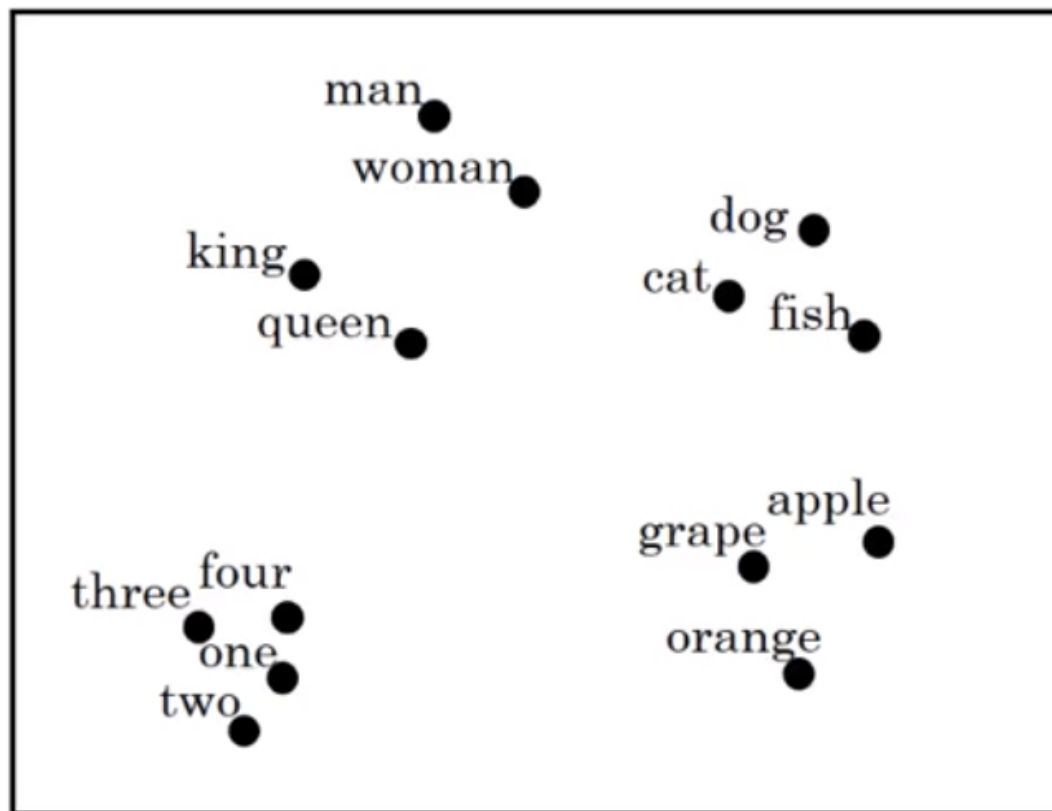
I want a glass of orange juice.

I want a glass of apple juice.

■ 优点:

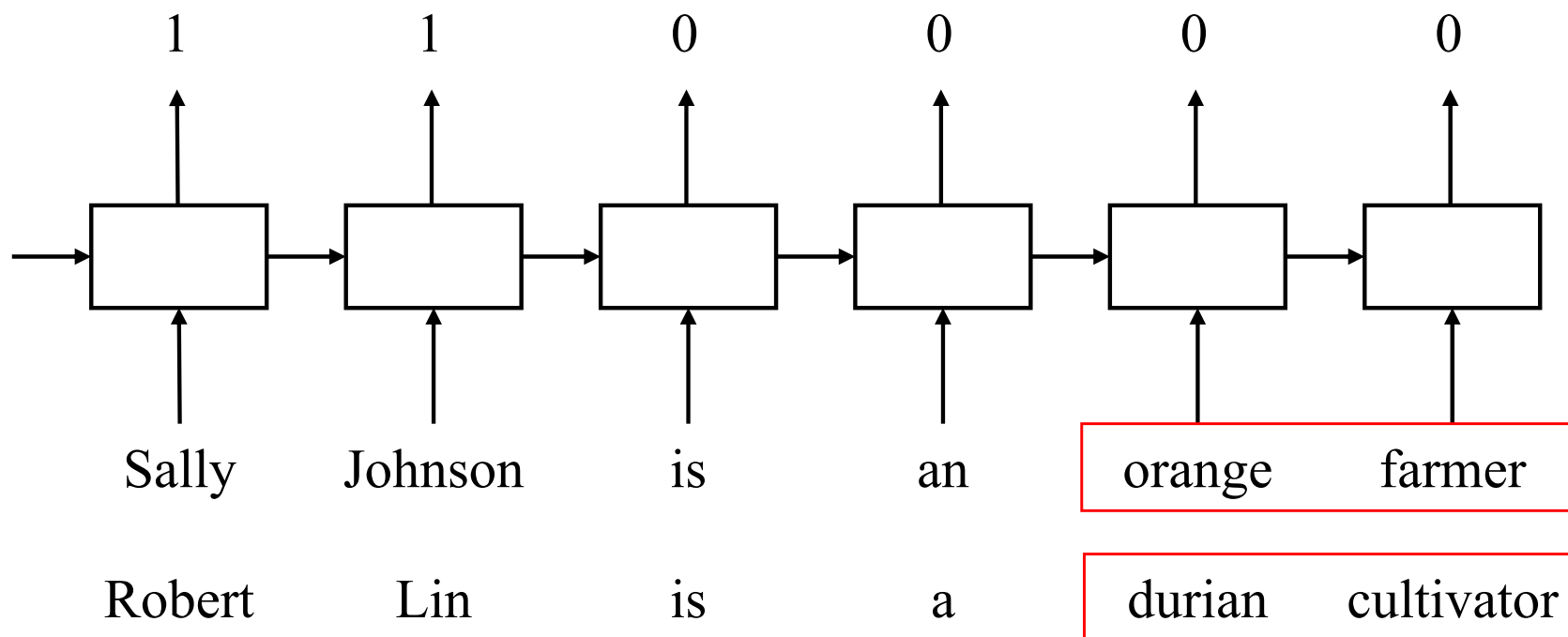
- 根据特征向量能清晰知道不同单词之间的相似程度
- 提高了有限词汇量的泛化能力

■ 词嵌入(word embedding): 特征化表征



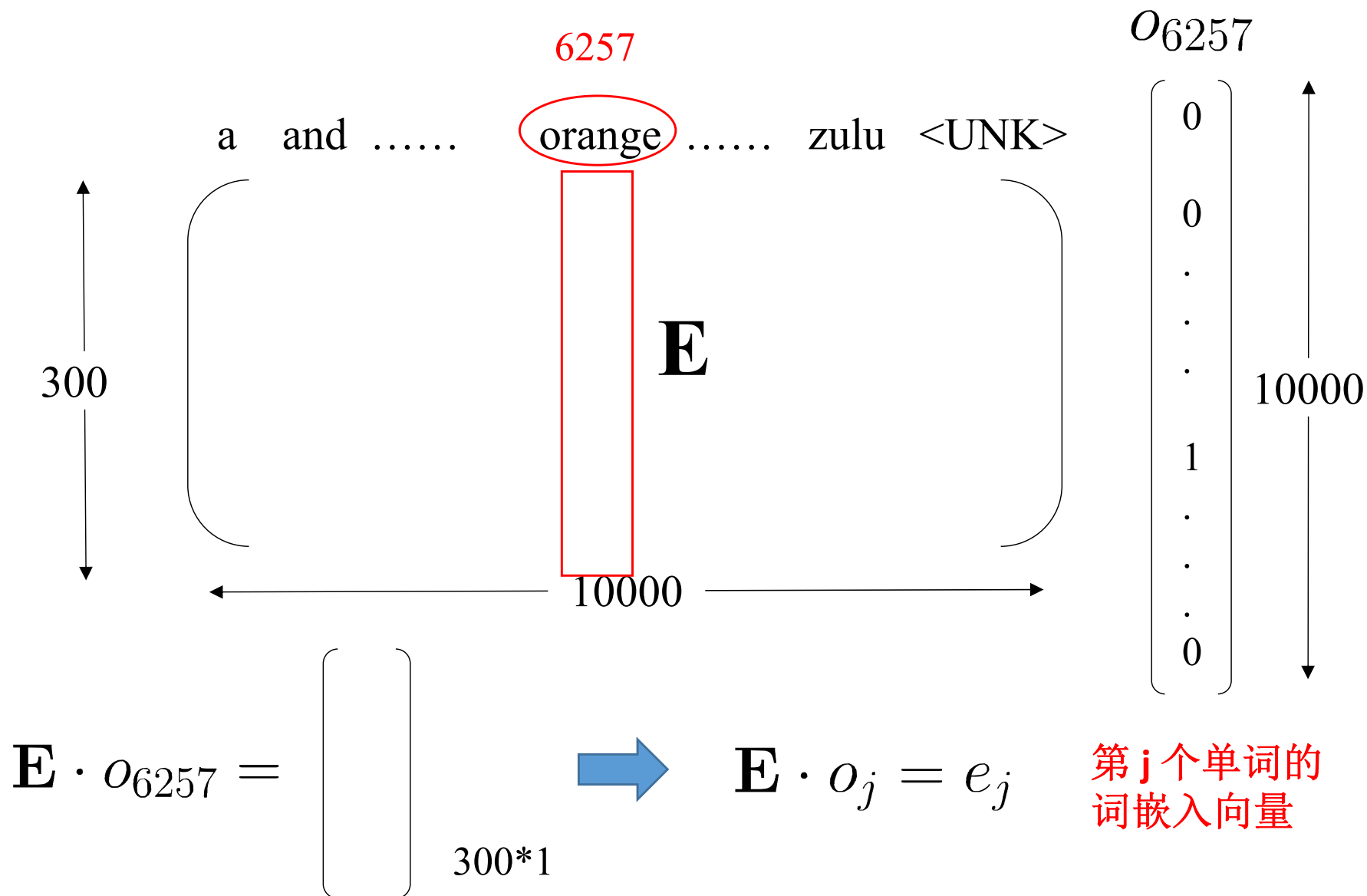
对于相近的概念，学习的特征比较类似

- 使用词嵌入做迁移学习：将从大量文本中学习的知识迁入到命名实体识别中



- 从海量词汇库中学习词嵌入。或者从网上下载预训练好的词嵌入。
- 在新任务中（样本量少）使用词嵌入模型。
- (optional): 在新任务数据上微调词嵌入模型。

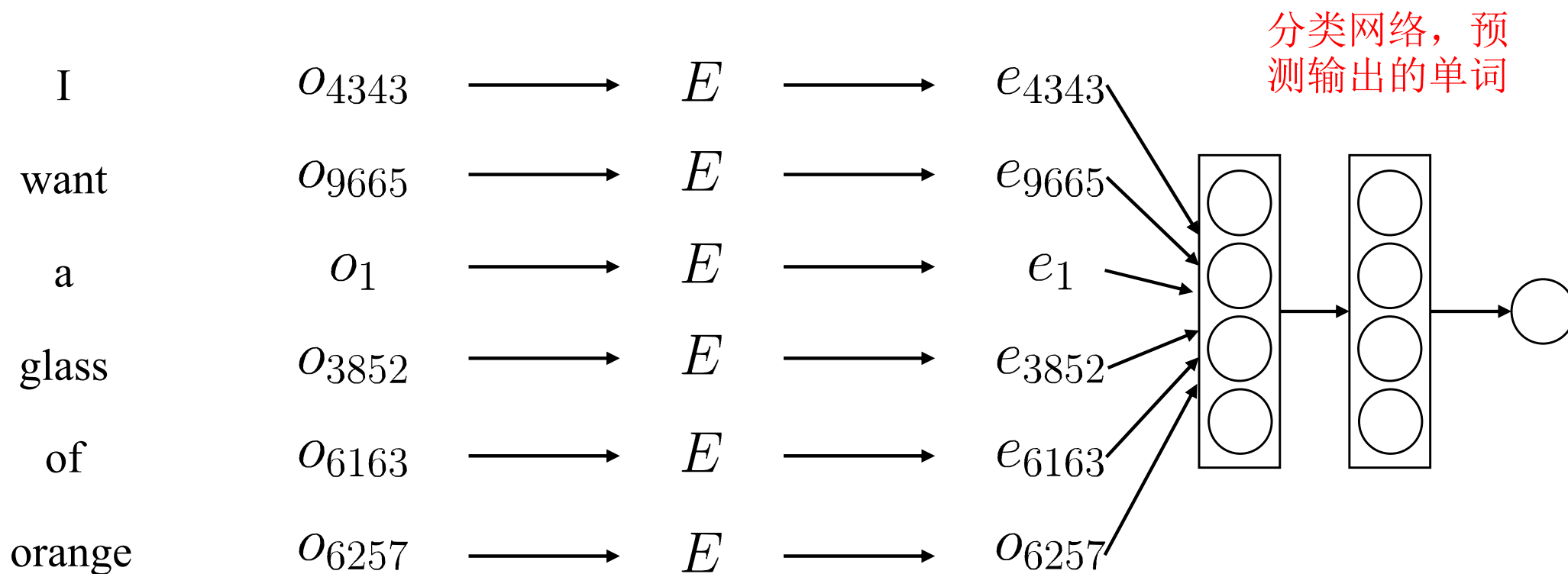
- 学习词嵌入：实际上是学习一个嵌入矩阵(embedding matrix)



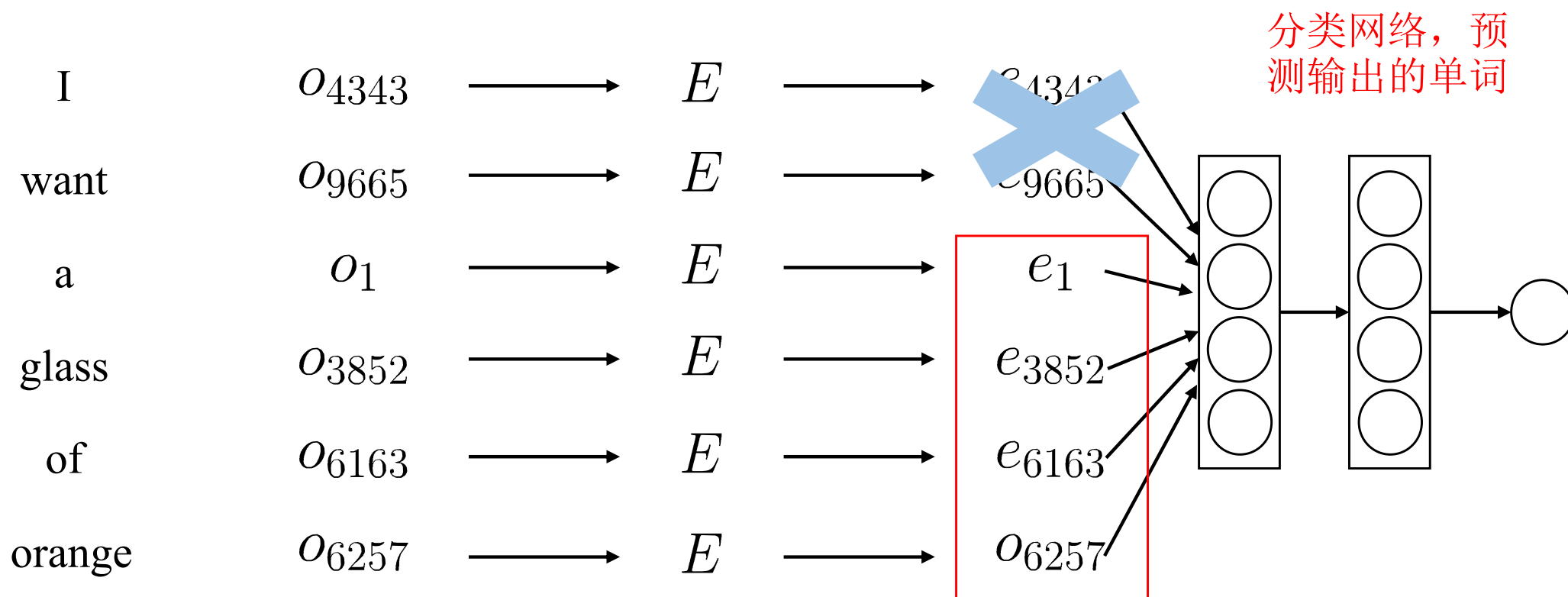
学习词嵌入：用语言模型学习词嵌入

I want a glass of orange _____.

4343 9665 1 3852 6163 6257



- 学习词嵌入：用语言模型学习词嵌入
- 需训练的参数有 E 和分类网络的参数，可用梯度下降法优化求解



用固定窗口，算法可以处理任意长度的句子

- 学习词嵌入：用语言模型学习词嵌入
- 需训练的参数有E和分类网络的参数，可用梯度下降法优化求解
- 同类单词的嵌入特征相似

I want a glass of orange juice.

I want a glass of apple juice.

- 学习词嵌入：用语言模型学习词嵌入

I want a glass of orange juice to go along with my cereal.

context Target

- 上下文：
 - 前n个单词，后n个词
 - 前1个单词
 - 附近某1个单词(Skip-Gram)

■ Skip-grams → 学习词嵌入模型

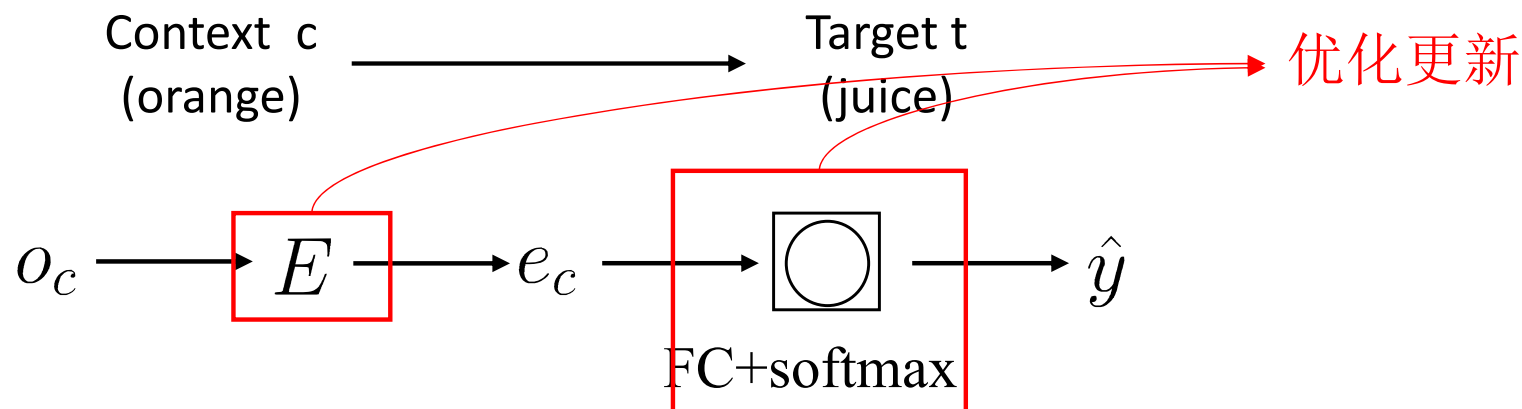
- 随机选择一个词作为context
- 随机在一定词距中选择一个单词作为target
- 给定context单词，预测target单词

I want a glass of orange juice to go along with my cereal.

	context	Target
样本1:	orange	juice
样本2:	orange	glass
样本3:	orange	my

■ Skip-grams → 学习词嵌入模型

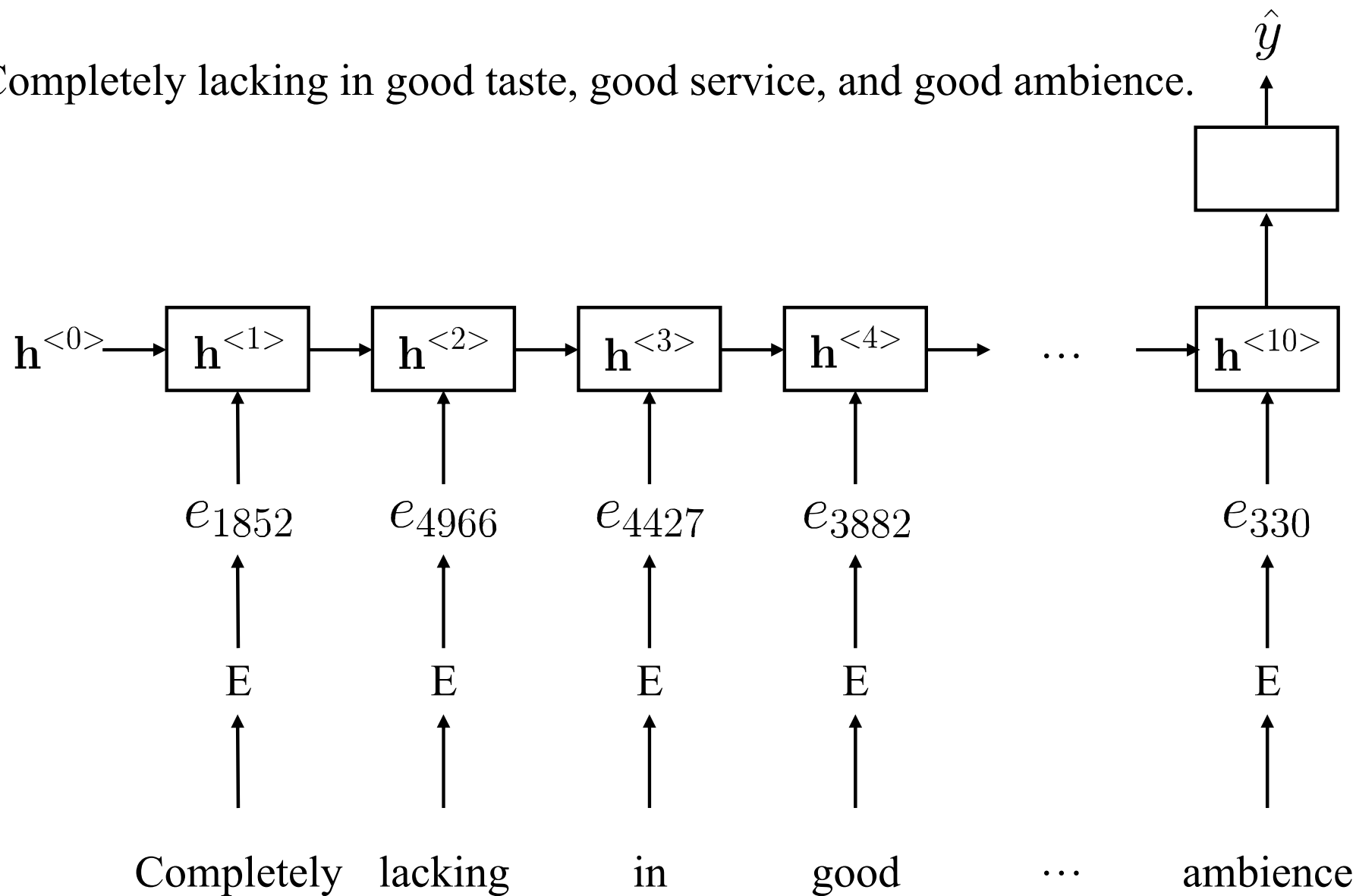
- 随机选择一个词作为context
- 随机在一定词距中选择一个单词作为target
- 给定context单词，预测target单词



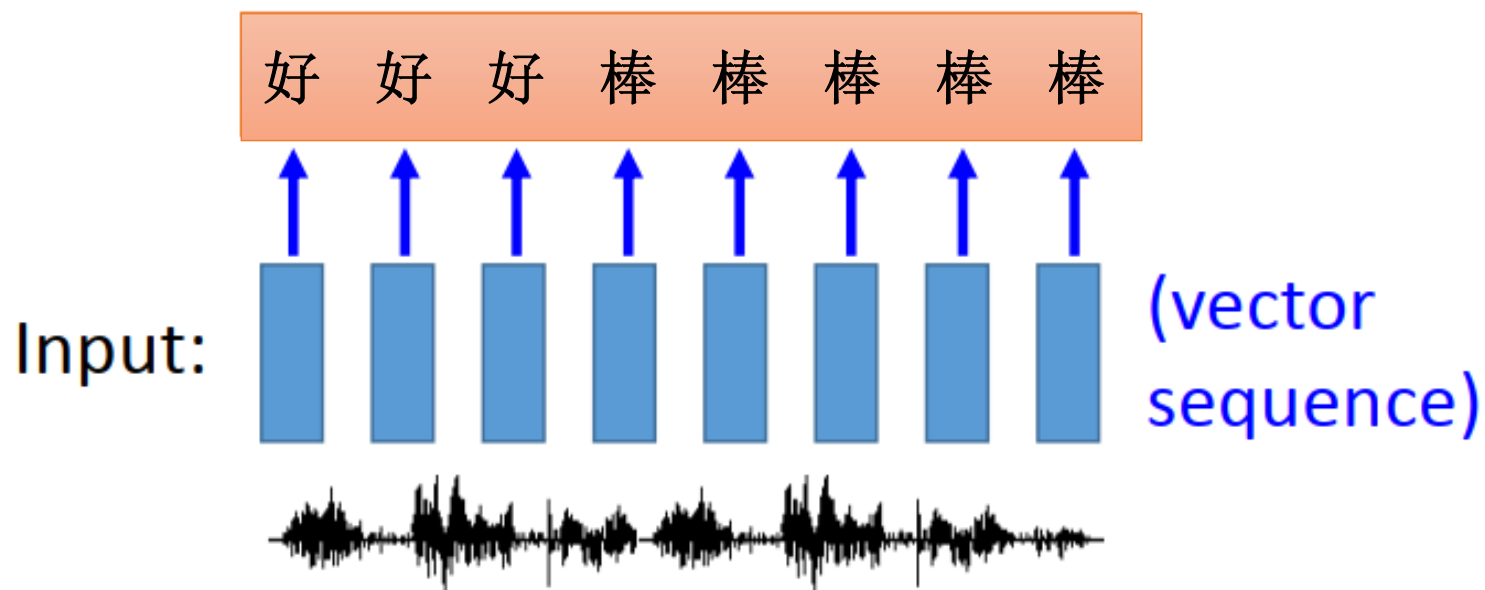
FC + softmax:
$$p(t|c) = \frac{\exp(\theta^T e_c)}{\sum \exp(\theta_j^T e_c)}$$

$$L(\hat{y}, y) = \sum_{i=1}^{10000} y_i \log \hat{y}_i$$

Completely lacking in good taste, good service, and good ambience.



- 输入输出都是sequence，长度不一样



- 输入输出都是sequence，长度不一样

Output: “好棒” (character sequence)

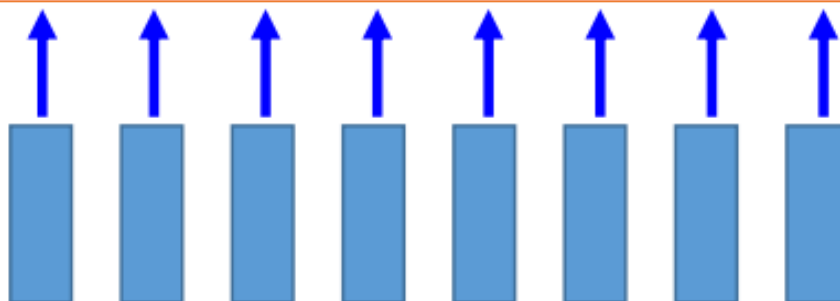


Trimming

问题：
如果想输出“好棒棒”怎么办？

好 好 好 棒 棒 棒 棒 棒

Input:

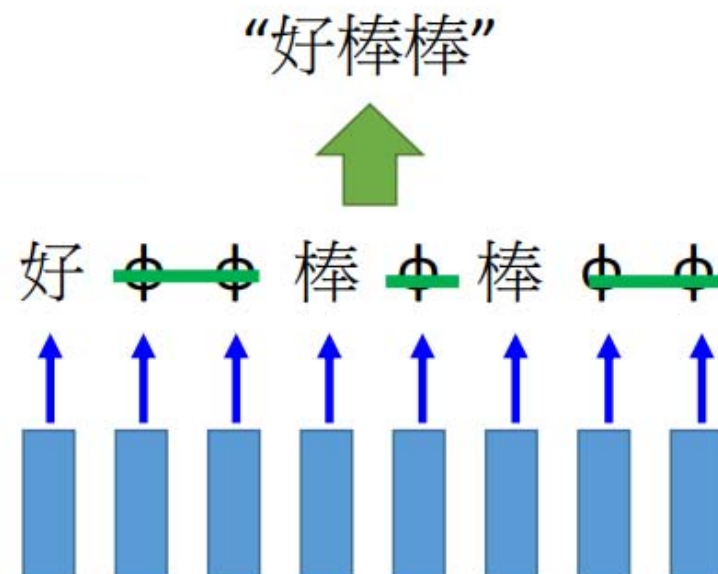
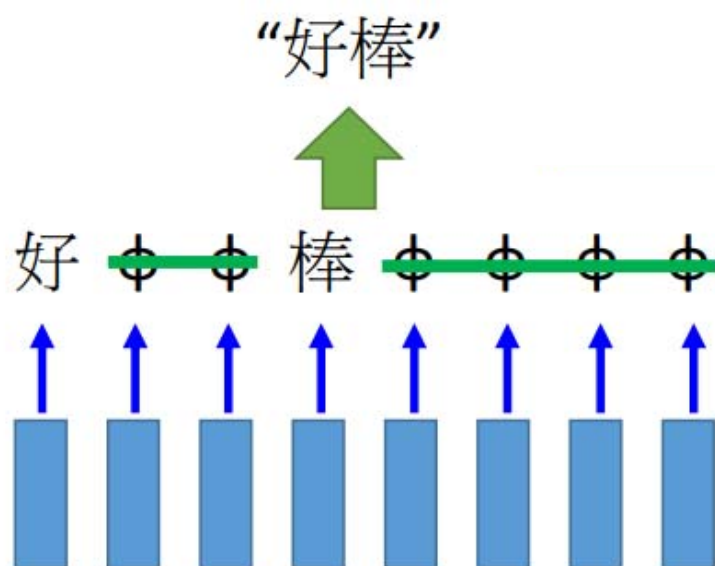


(vector
sequence)



- CTC (connectionist temporal classification) [Alex Graves, ICML'06][Alex Graves, ICML'14][Haşim Sak, Interspeech'15][Jie Li, Interspeech'15][Andrew Senior, ASRU'15]

加入额外的字符 ϕ 表示空集



- CTC (Connectionist Temporal Classification) [Alex Graves, ICML'06][Alex Graves, ICML'14][Haşim Sak, Interspeech'15][Jie Li, Interspeech'15][Andrew Senior, ASRU'15]

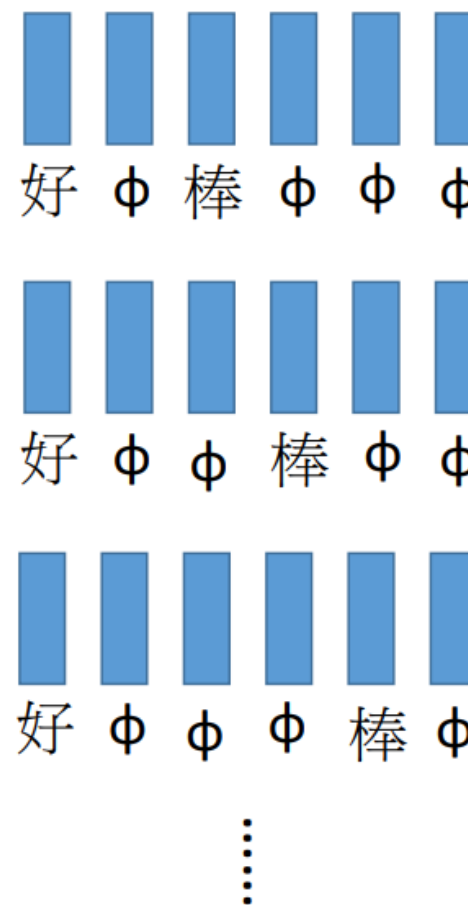
- 训练过程

Acoustic
Features:



Label: 好 棒

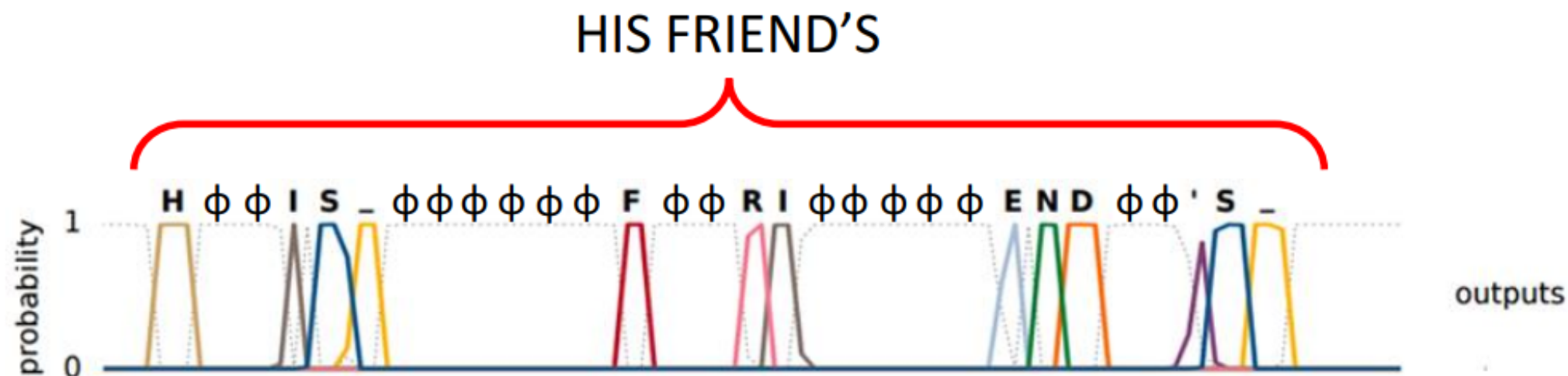
穷举所有可能的label作为训练样本



4 Sequence to sequence learning (Seq2Seq)

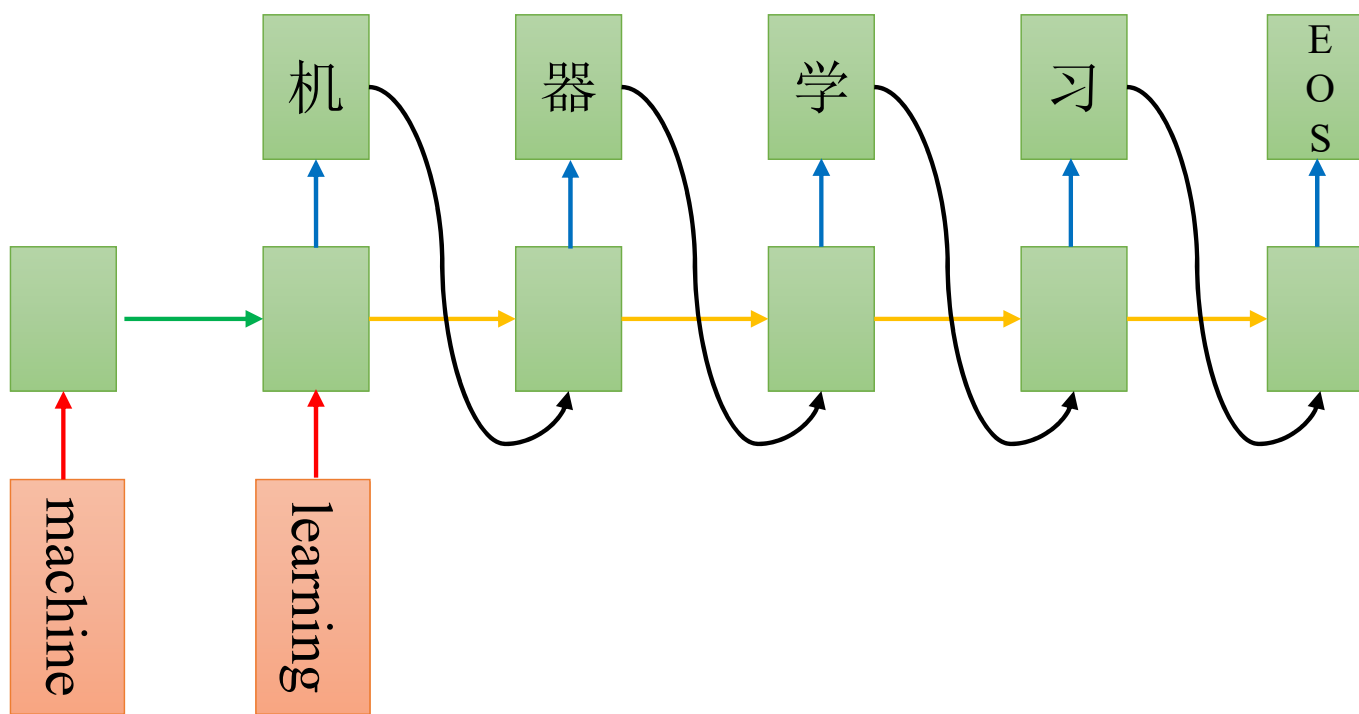
- CTC (Connectionist Temporal Classification) [Alex Graves, ICML'06][Alex Graves, ICML'14][Haşim Sak, Interspeech'15][Jie Li, Interspeech'15][Andrew Senior, ASRU'15]

- 测试过程



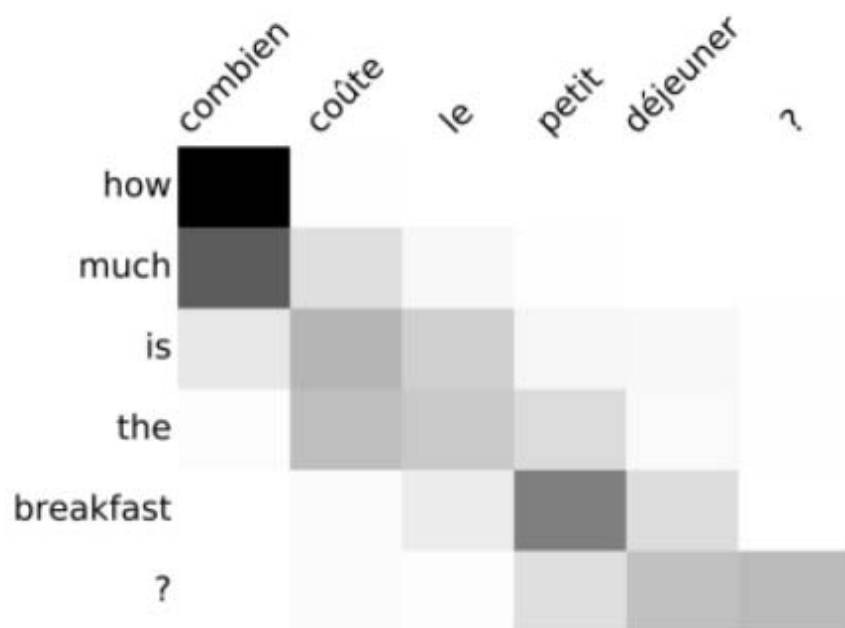
4 Sequence to sequence learning (Seq2Seq)

■ 机器翻译 (machine translation)

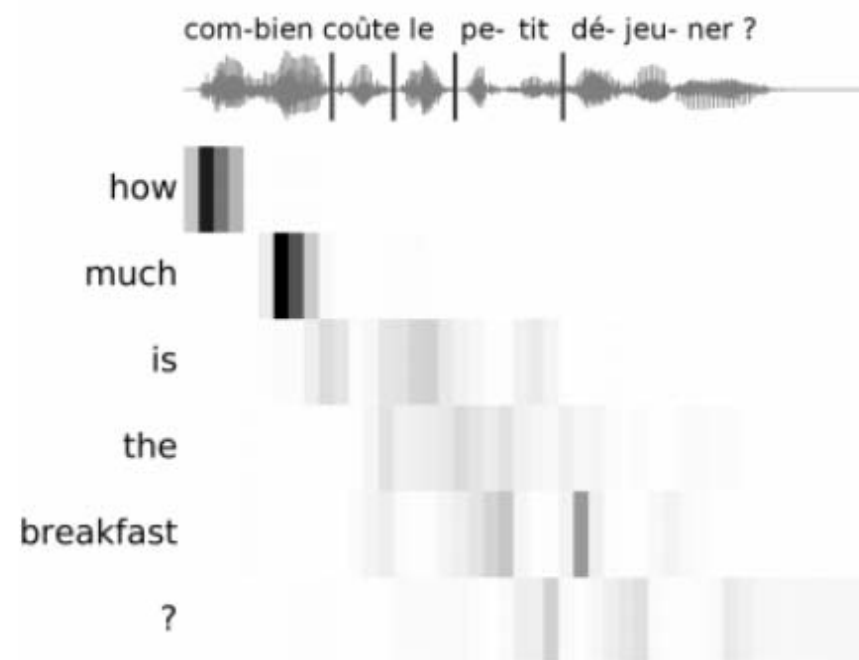


4 Sequence to sequence learning (Seq2Seq)

- 机器翻译 (machine translation): A语言音频 $\rightarrow$ B语言文字



(a) Machine translation alignment

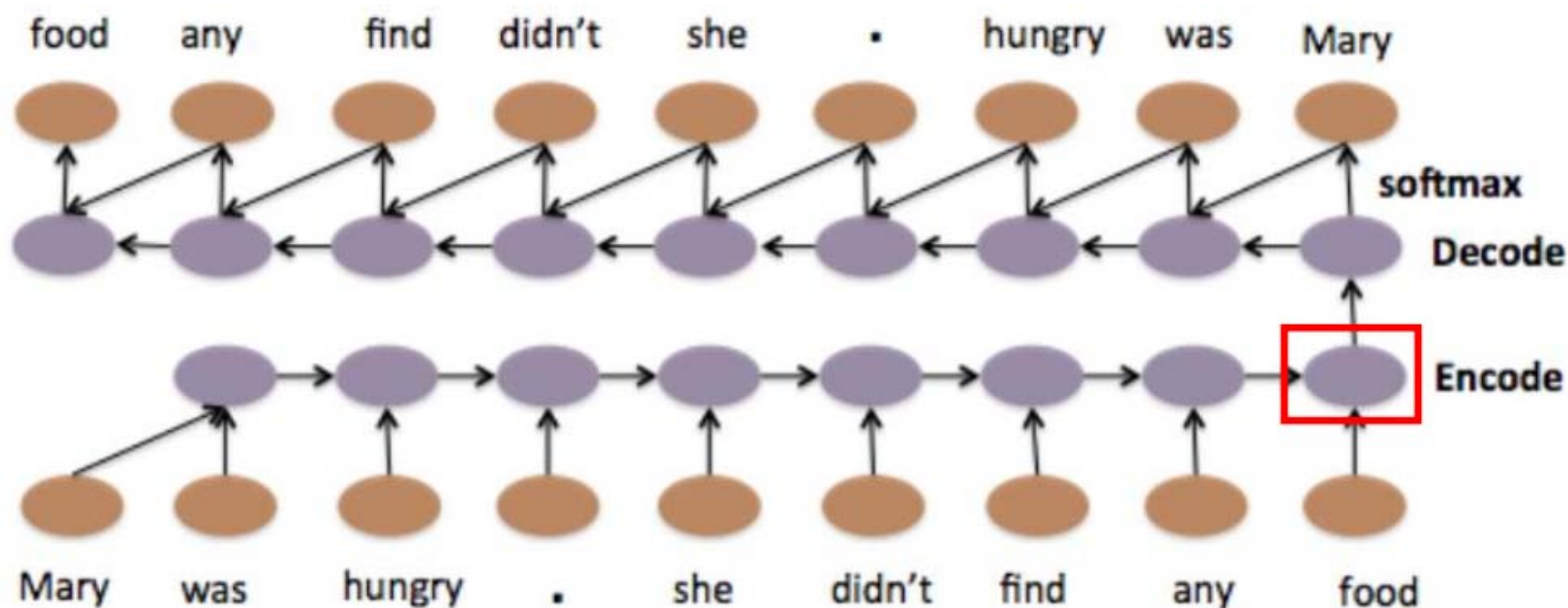


(b) Speech translation alignment

4 Sequence to sequence learning (Seq2Seq)

■ Auto-encoder (自动编码) → 提取文本特征

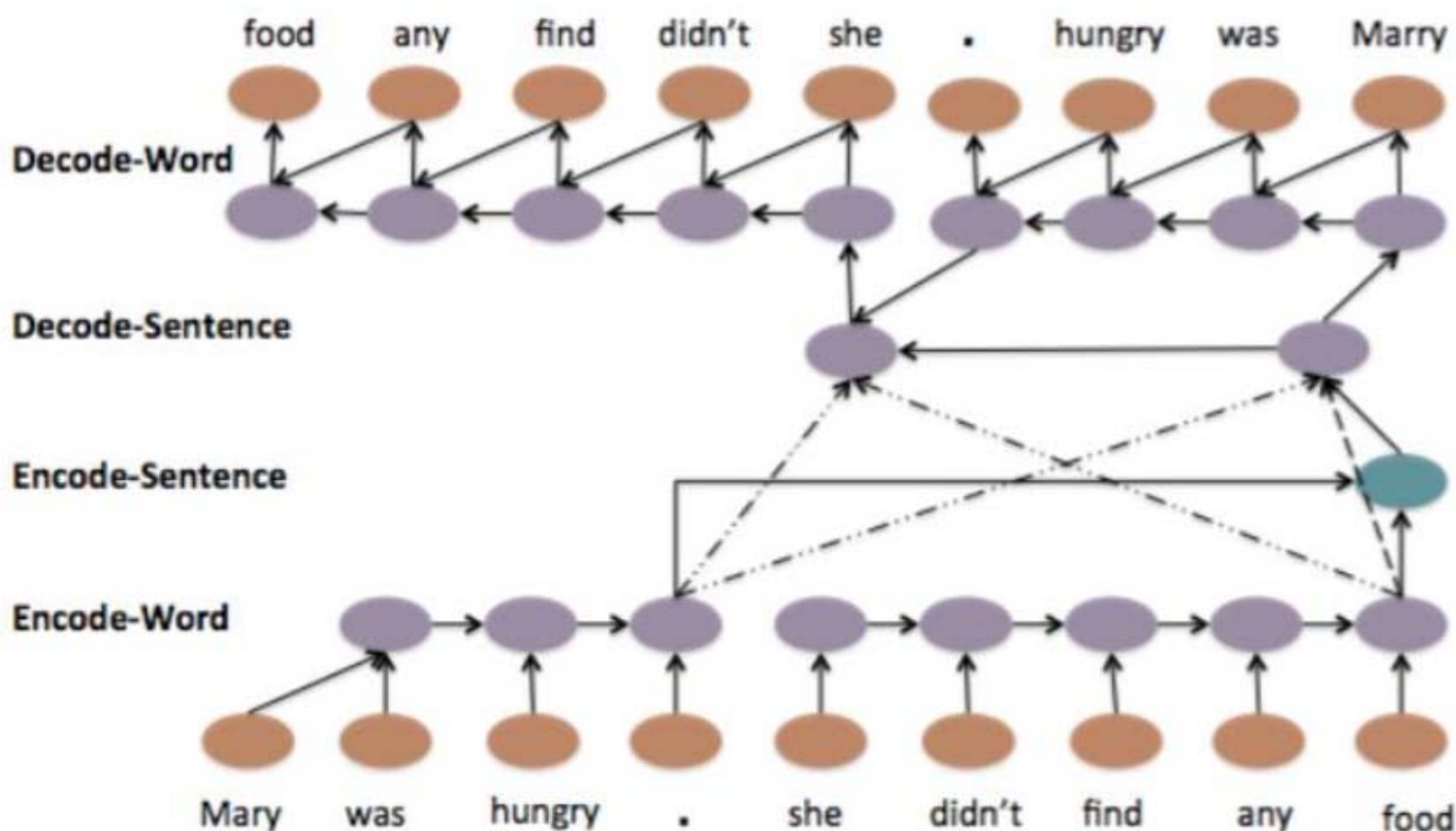
□ 用RNN编码将文本编码成向量，再用RNN 解码成原文本



4 Sequence to sequence learning (Seq2Seq)

■ Auto-encoder (自动编码) → 提取文本特征

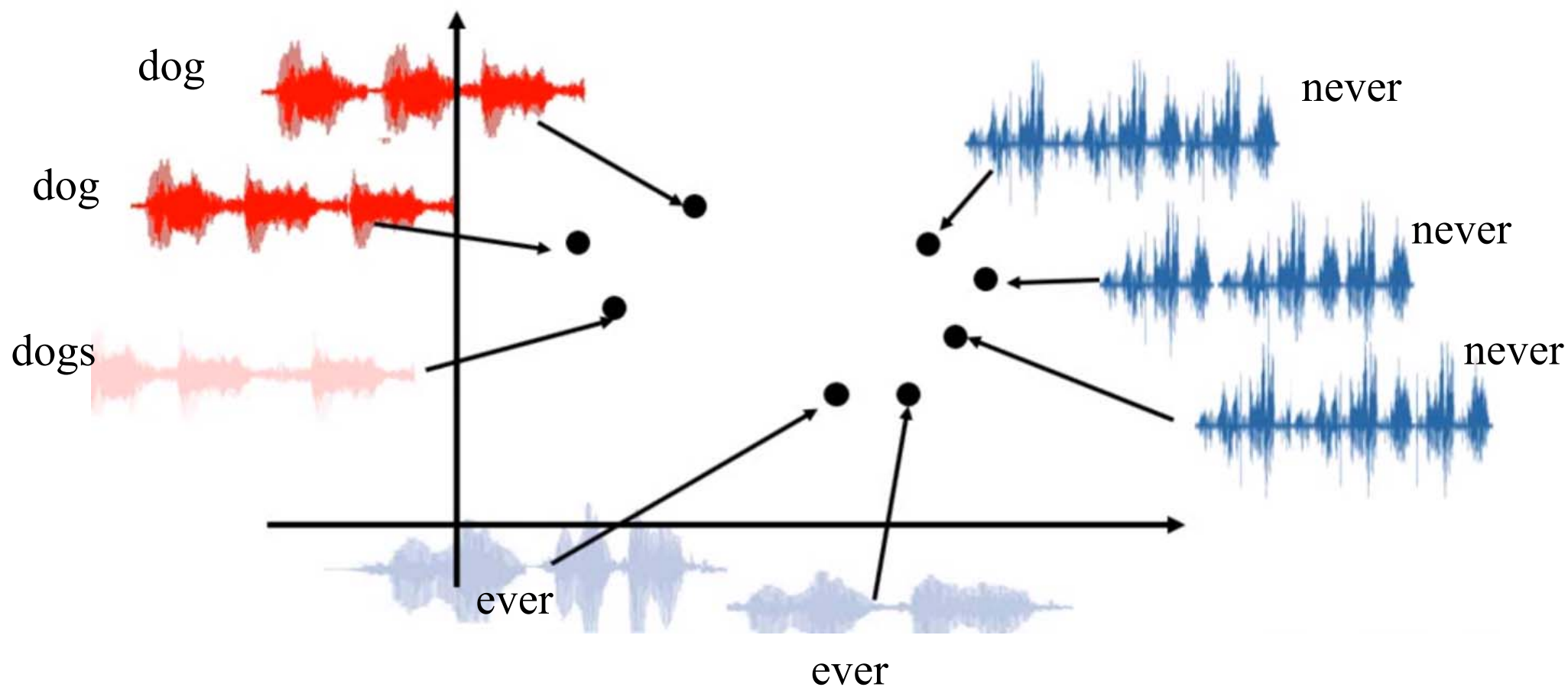
□ Hierarchical: 每个句子提取特征 → 整合成文档特征 → 解码成每个句子的特征 → 解码成单词序列



4 Sequence to sequence learning (Seq2Seq)

■ Auto-encoder (自动编码) → 提取语音特征

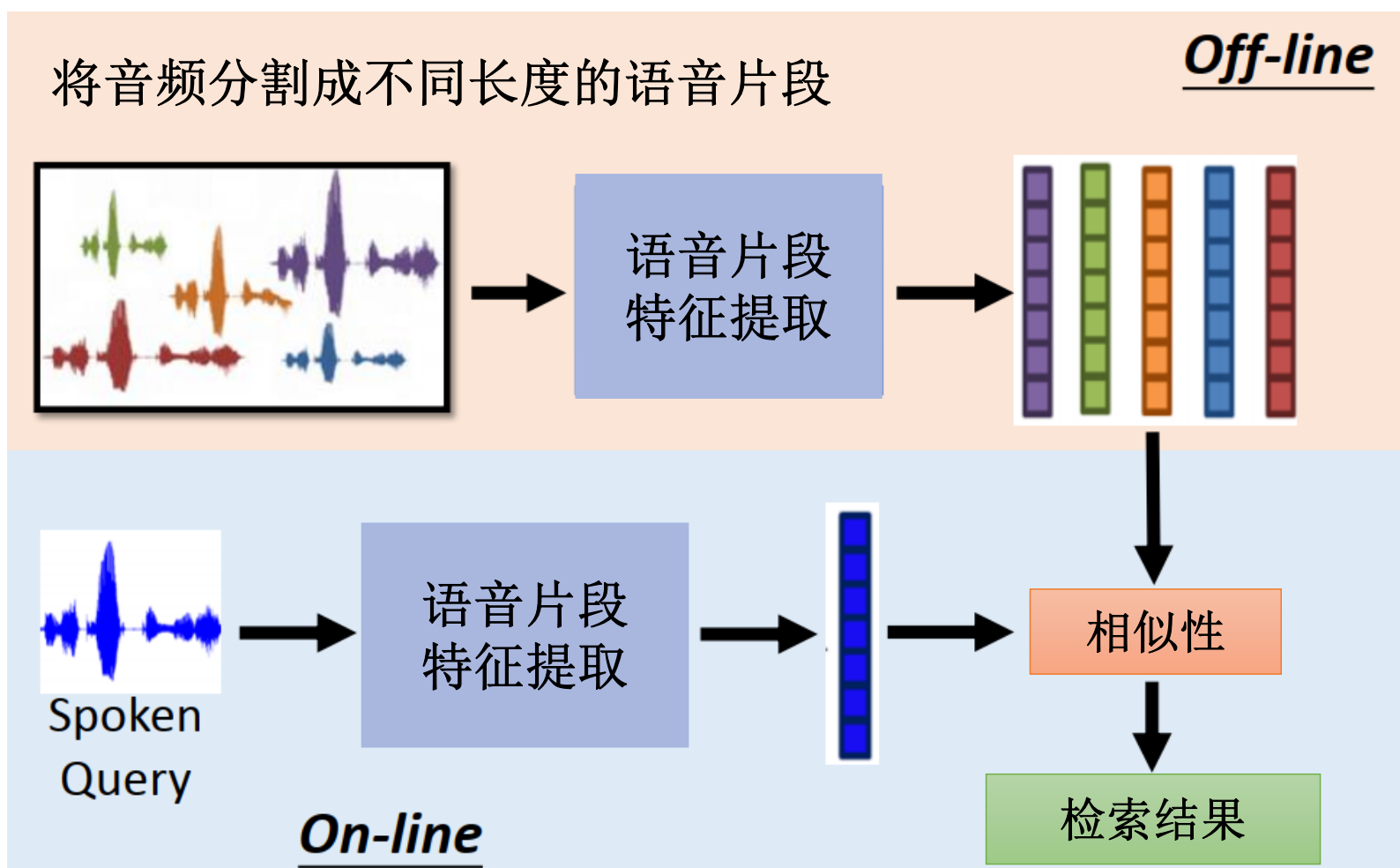
□ 不同长度的语音信号 → 固定长度的特征向量



4 Sequence to sequence learning (Seq2Seq)

■ Auto-encoder (自动编码) → 提取语音特征

□ 语音查询

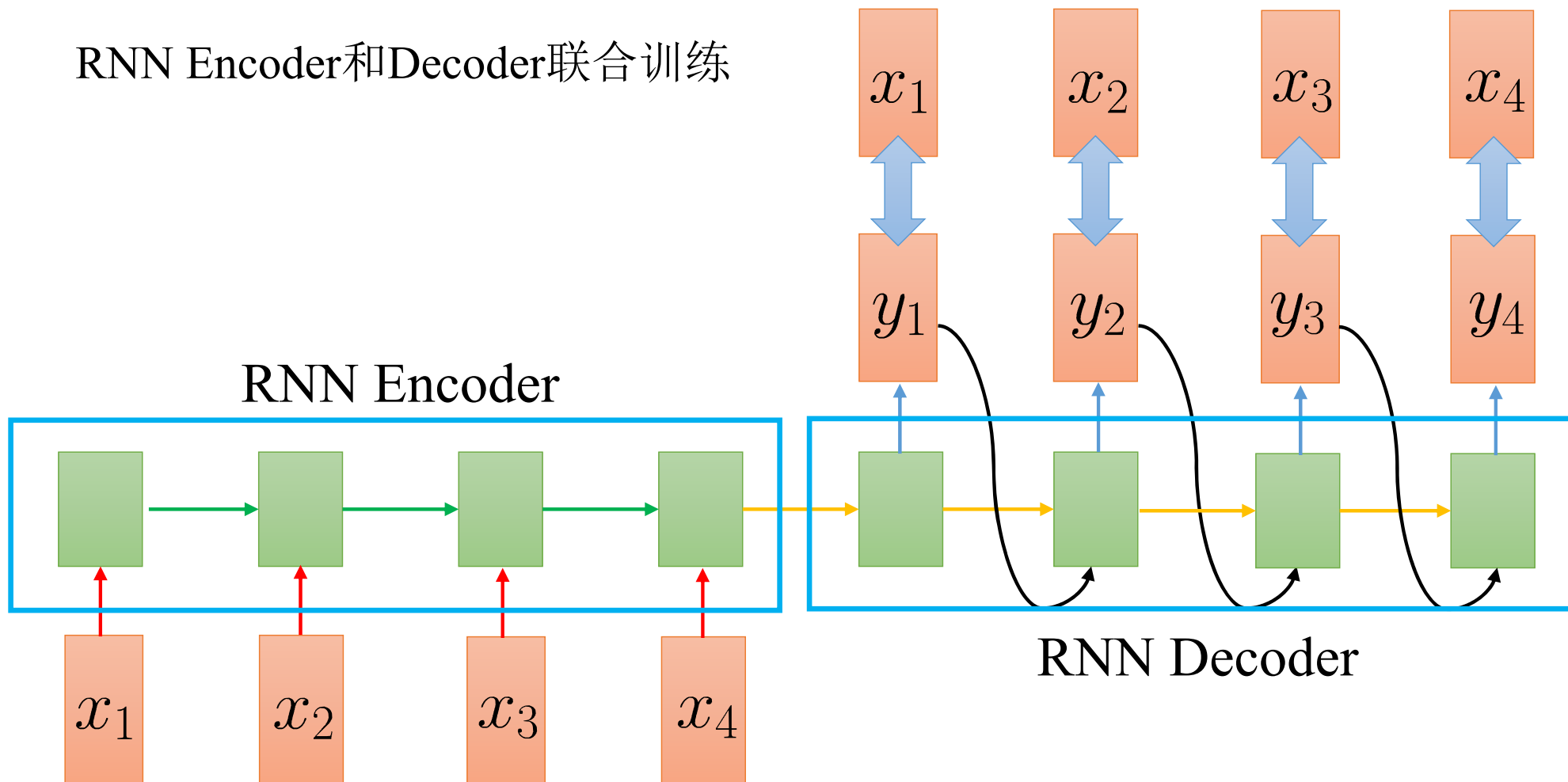


4 Sequence to sequence learning (Seq2Seq)

■ Auto-encoder (自动编码) → 提取语音特征

□ 语音特征提取

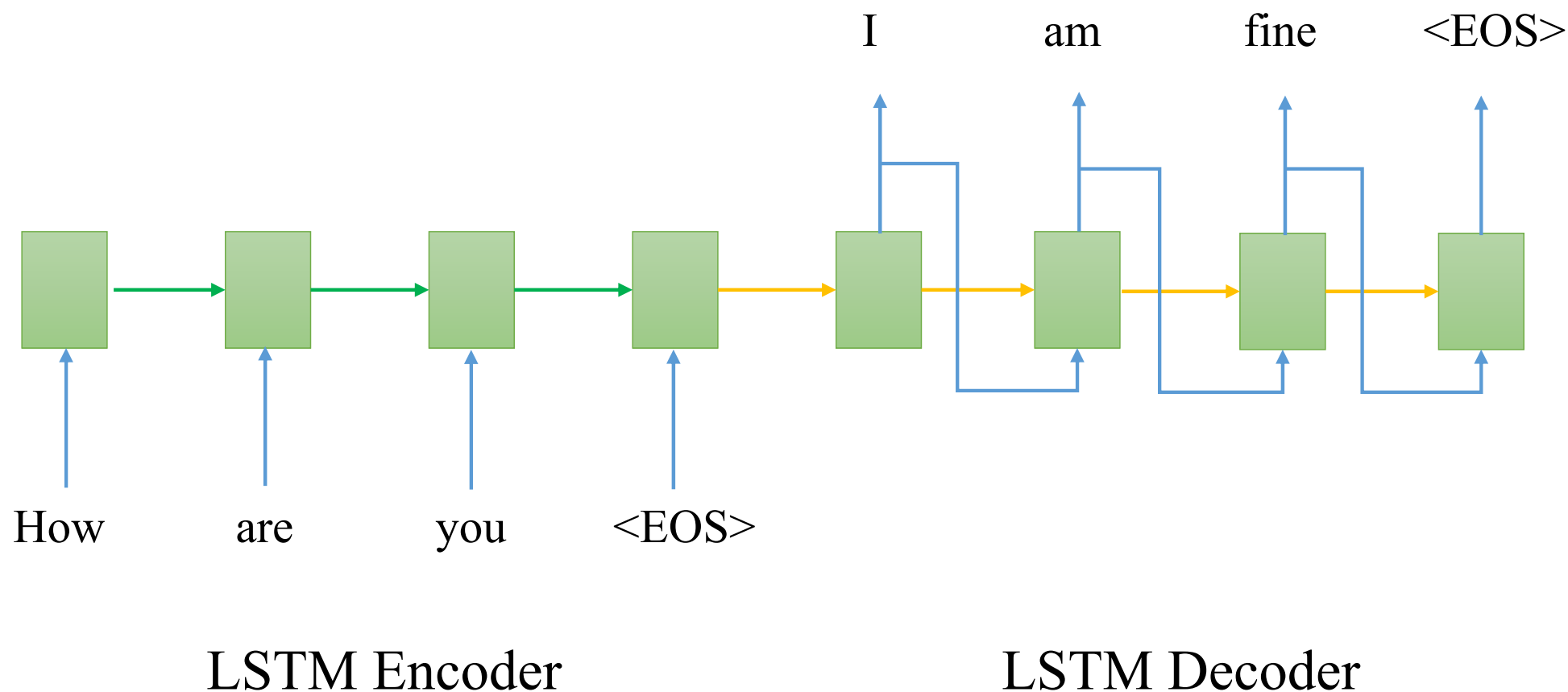
RNN Encoder和Decoder联合训练



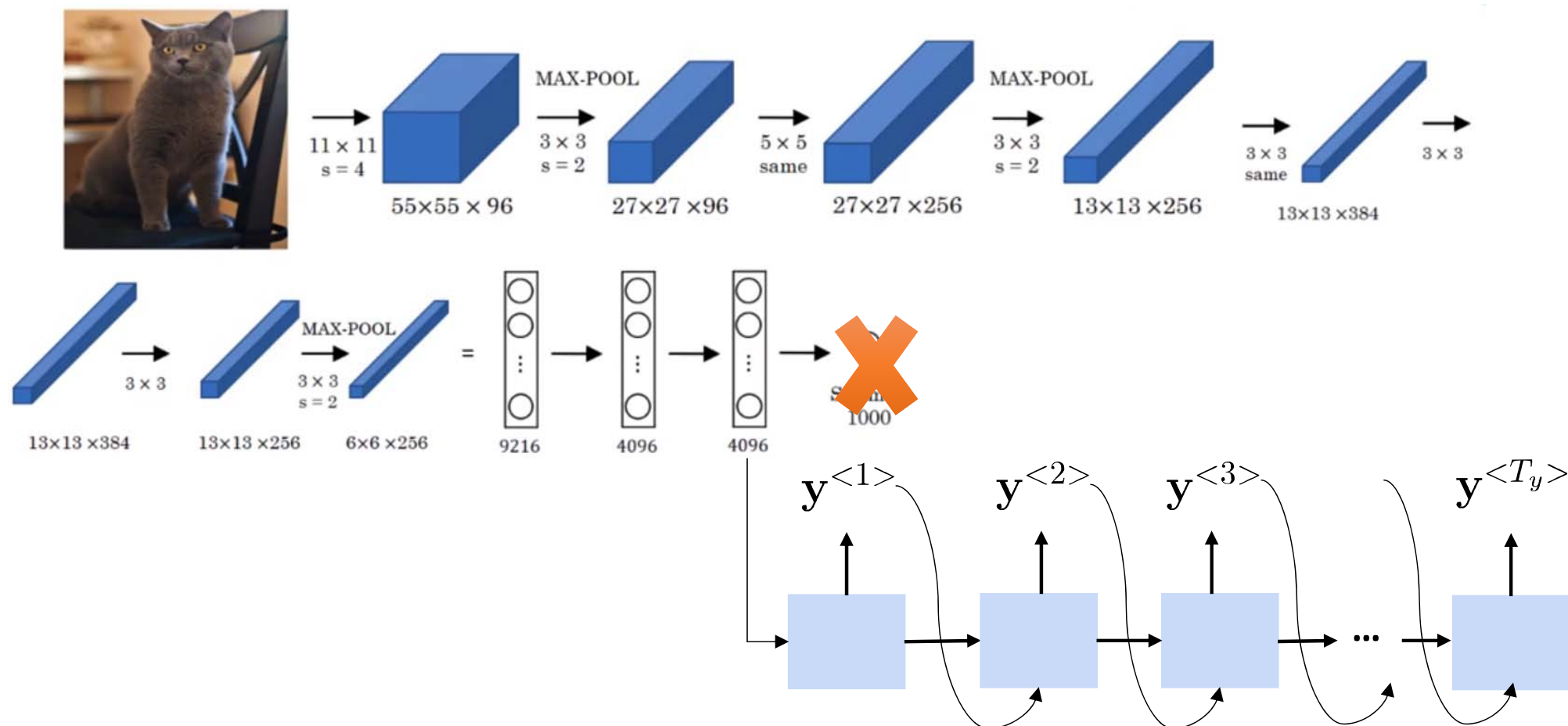
4 Sequence to sequence learning (Seq2Seq)

■ Auto-encoder (自动编码)

□ Chat-bot



$\mathbf{y}^{<1>}$ **$\mathbf{y}^{<2>}$** **$\mathbf{y}^{<3>}$** **$\mathbf{y}^{<4>}$** **$\mathbf{y}^{<5>}$** **$\mathbf{y}^{<6>}$**
A cat sitting on a chair



Deepjazz: <https://github.com/jisungk/deepjazz>

原音乐



生成的音乐



1 epoch



16 epochs



32 epochs



64 epochs

04

PART FOUR

小结

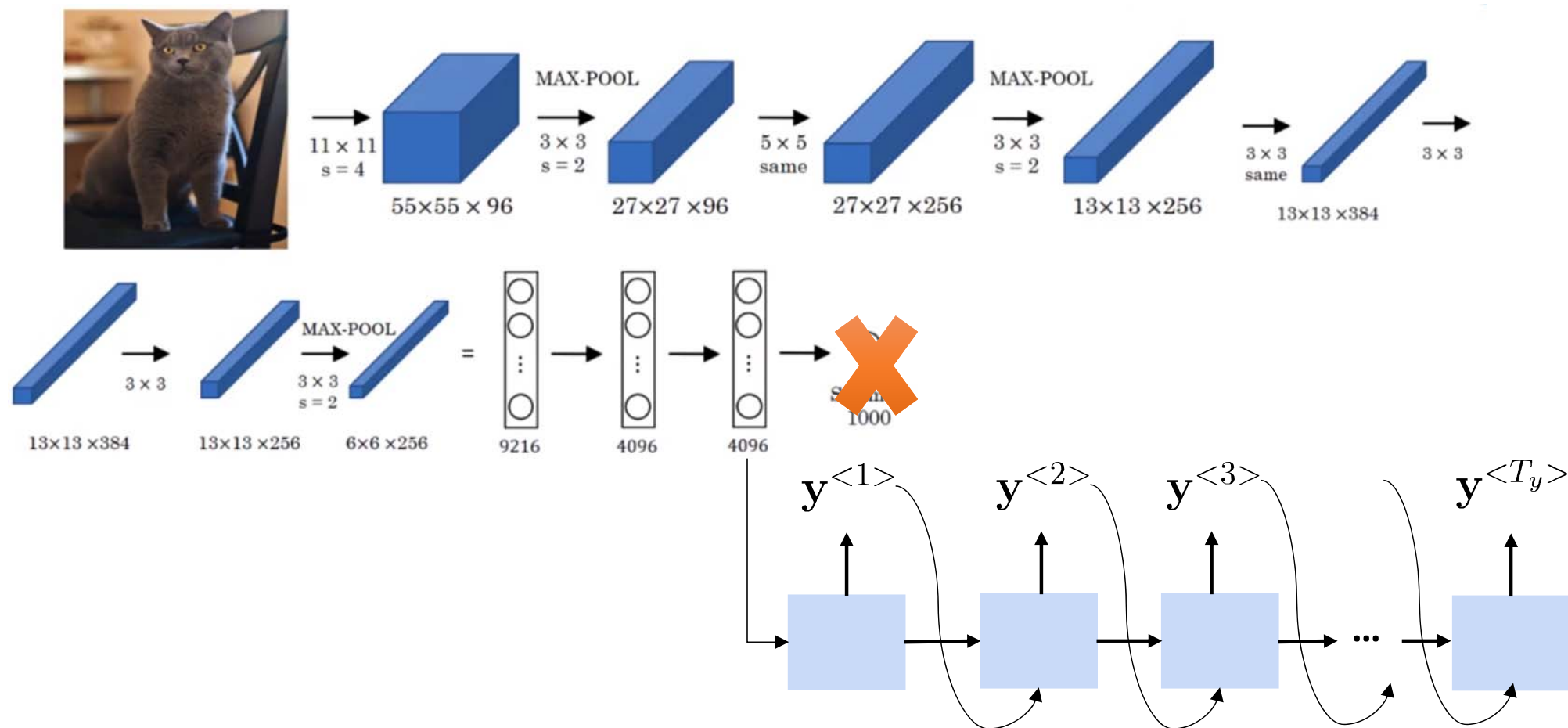
Conclusions

- LSTM
- GRU
- RNN的应用
 - ☐ 词嵌入
 - ☐ 情感分类
 - ☐ 语音识别
 - ☐ Seq2Seq
 - ☐ Auto-encoder
 - ☐ 图像加字幕
 - ☐ 音乐生成

1 Image Captioning

$y^{<1>} \quad y^{<2>} \quad y^{<3>} \quad y^{<4>} \quad y^{<5>} \quad y^{<6>}$

A cat sitting on a chair



4

触发字检测



Amazon Echo
(Alexa)



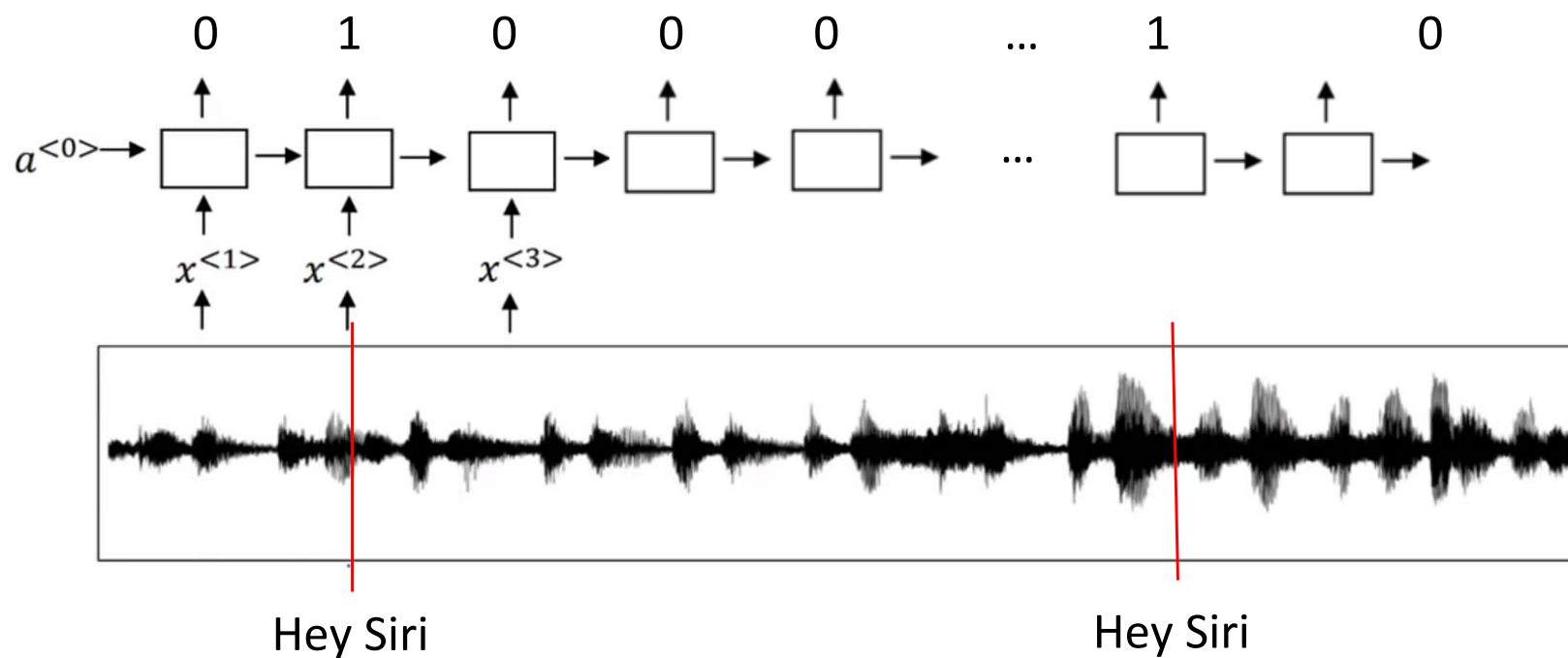
Baidu DuerOS
(xiaodunihao)



Apple Siri
(Hey Siri)



Google Home
(Okay Google)



04

PART FOUR

小结和资源

Take Home Message and Resource

- 在周杰伦歌词数据集上，应用循环神经网络训练语言模型，并利用该模型来创作歌词。